SOFTWARE REQUIREMENTS SPECIFICATION

DevWeave

AI Software Delivery Control Plane: Hybrid Agent Coordination and Project Knowledge

Control	Value
Document ID	DWV-SRS-001
Version	1.5
Status	Business-aligned baseline for product, engineering and commercial model review; formal product-owner and engineering approval pending. Includes agency-first positioning, external-agent coordination and optional managed execution.
Prepared	09 November 2026
Product owner	To be assigned
System owner	To be assigned
Classification	Project planning document
Normative baseline	Product-specific requirements; standards are guidance unless adopted by contract

Plan Together. Build Together. Preserve Everything.

Baseline statement

This SRS defines the intended product boundary and an implementable first architecture. It is a review baseline, not evidence that the system has been implemented, security-certified, or capacity-tested.

Prepared from the product concept and architecture decisions established in the preceding analysis.

Document Control

Version	Date	Owner	Change summary	Status
0.1	09 Nov 2026	Product analysis	Initial domain and architecture outline	Draft
1.0	09 Nov 2026	Product analysis	Complete SRS baseline, extensibility, acceptance and delivery model	Proposed baseline
1.1	09 Nov 2026	Product analysis	Added distributed multi-agent coordination over shared external codebases; plan baselines, remote agent runtimes, isolated workspaces, leases, integration queue, recovery and acceptance criteria	Proposed amendment
1.2	09 Nov 2026	Product analysis	Added project-scoped MCP service and durable real-time agent collaboration: discovery, presence, channels, mediated messages, resource subscriptions, event replay, cursor recovery, protocol conformance and acceptance criteria.	Proposed amendment
1.3	09 Nov 2026	Product analysis	Applied DevWeave product branding consistently across the SRS, document identifiers, coordination tool names and MCP resource namespace; retained the project-scoped real-time agent coordination requirements.	Proposed baseline
1.4	09 Nov 2026	Product analysis	Consolidated final DevWeave baseline; clarified mandatory logical project MCP service for every project, resolved optional-adapter ambiguity, aligned agent interfaces and acceptance conditions, and incorporated the published MCP 2026-07-28 stateless/subscription interaction model.	Consolidated baseline for review
1.5	09 Nov 2026	Product analysis	Aligned product vision and release plan to AI Software Delivery Control Plane; agency-first market entry; agent-neutral customer-managed execution; optional DevWeave-managed execution; commercial entitlements, usage metering, client access and product success measures.	Proposed business-aligned baseline

Review and approval

Role	Name	Decision	Date
Product owner	TBD	Pending	TBD
Engineering lead	TBD	Pending	TBD
Security/privacy reviewer	TBD	Pending	TBD
Operations owner	TBD	Pending	TBD

Document status and change control

This document is a proposed baseline. Changes to scope, architecture constraints, requirement priority, retention policy, security boundary, or acceptance targets shall be recorded as change requests. The product owner and engineering lead shall approve changes that affect the MVP baseline. Security, privacy and operational requirements require review by the responsible specialist before release.

How to use this SRS

- Requirement IDs are stable identifiers and shall be used in design notes, implementation tickets, test cases and release evidence.
- “Shall” denotes a mandatory requirement; “should” denotes a recommendation requiring recorded rationale if deferred; “may” denotes an optional capability.
- Priority: M = Must, S = Should, C = Could. Release: MVP = foundation baseline; R1 = first commercial pilot/launch, adding external multi-agent coordination; R2 = hybrid managed-execution and scale release; R3/FUT = enterprise and advanced intelligence capabilities.
- Acceptance statements define the minimum observable result; detailed procedures belong in the verification plan and test suite.
- Values labelled “initial target” are planning targets and must be validated against measured workload before production commitments.

Contents

1. Introduction
2. Overall Product Description
3. Product Architecture and Design Constraints
4. Stakeholders, Actors and Use Cases
5. External Interface Requirements
6. Functional Requirements
7. Information and Data Requirements
8. Non-Functional Requirements
9. Business Rules and Invariants
10. Security, Privacy and AI Governance
11. Verification and Acceptance
12. Deployment, Operations and Support
13. Release Scope and Delivery Strategy
14. Traceability and Requirement Governance
15. Assumptions, Dependencies, Risks and Open Decisions
16. Multi-Agent Distributed Execution and Shared-Codebase Coordination
17. Project-Level MCP Service and Real-Time Agent-to-Agent Coordination
18. Business Vision, Commercial Model and Business-Aligned Requirements

Appendix A. Initial Knowledge Type Catalogue
Appendix B. Initial Relationship Type Catalogue
Appendix C. Canonical Data Concepts and Examples
Appendix D. Glossary
Appendix E. References and Standards
Appendix F. Coordination Data Concepts and State Models
Appendix G. Project MCP Service Contracts and Message Flow
Appendix H. Commercial Product Terms

Heading styles are applied throughout. Refresh the table of contents in Word after opening; Section 18 and Appendix H contain the business-aligned product requirements and commercial terminology.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the business-aligned product, functional, information, interface, security, quality, operational, commercial-platform and extensibility requirements for DevWeave. It is the shared baseline for product planning, architecture, implementation, procurement, verification, release decisions and future change control. It distinguishes product capabilities and measurable business outcomes from non-binding market, pricing and revenue hypotheses.

DevWeave is an AI Software Delivery Control Plane for organizations that plan, coordinate and verify software development performed by humans and AI agents. It combines a persistent project-knowledge system with a project-scoped, real-time agent coordination service. It allows multiple agents operating across providers, devices, locations and runtime environments to execute an approved plan against one authoritative external codebase through isolated workspaces and governed integration. Customers may connect externally operated agents and, optionally, use DevWeave-managed execution environments. The canonical knowledge store is not a source-code repository: managed runtimes may temporarily materialize source code in isolated, ephemeral workspaces, but source code shall not be retained as project knowledge or permanent platform content.

1.2 Product vision

Product vision / DevWeave shall enable software organizations—initially AI-first software development agencies and AI-native studios—to turn approved software plans into coordinated, evidence-backed delivery across multiple AI agents and execution environments. It shall preserve the project knowledge and decision history, coordinate external agents through a project-level MCP service, provide optional managed execution, and expose trustworthy progress and delivery evidence to authorized owners, clients, reviewers and other stakeholders. Its long-term direction is to serve as an agent-neutral control plane across software projects, coding tools, repositories and execution providers.

1.3 Scope

The system shall provide a shared, extensible SaaS foundation on which each tenant can create and manage multiple software-development projects. Each project may represent a product, system, application suite, platform or software initiative and may reference multiple repositories, services, deployment environments, agent providers and external tools. Each project shall have a logical project-scoped MCP service identity and resource namespace for authorized real-time agent coordination. The platform shall support customer-managed external agent execution as the initial commercial mode and optional DevWeave-managed execution as a later, separately metered mode. Both modes shall use the same canonical project knowledge, coordination, governance and evidence services.

1.3.1 In scope

- Multi-tenant identity, membership, project ownership, project-scoped access grants and tenant application installations.
- A universal knowledge-object model with registered types, versioned schemas, metadata, revisions, typed relationships and provenance.
- Lifecycle and execution records for planning, design, AI/human activities, verification, release, deployment, operation and evolution.
- Documents, diagrams, charts, structured artifacts, evidence and external references, subject to content policy and retention settings.
- AI-agent identity, authorization, context retrieval, activity reporting, output registration, evidence submission and review/approval.
- Stakeholder-specific dashboards, reports, search and learning/user views over a common canonical model.
- Integrations with source-code hosts, coding agents, CI/CD, testing, deployment, identity and other external systems, primarily through metadata and references. Distributed agent coordination, work-package scheduling, workspace/branch references, task/resource leases and merge/integration governance.
- Export, audit, schema evolution, operational monitoring and future application/extension capabilities.

1.3.2 Out of scope

- Permanent source-code hosting, source-code repository replacement, or retention of source files, diffs, patches, executable snippets or raw code-bearing transcripts in the canonical knowledge store. Optional DevWeave-managed execution may temporarily materialize source code in isolated, short-lived workspaces solely to perform authorized tasks; those workspaces are execution infrastructure, not project knowledge storage, and shall follow controlled cleanup and retention policies.

- Replacing Git hosting, IDEs, build systems, container registries, test execution platforms, deployment platforms or production observability tools.
- A general-purpose workflow automation engine unrelated to software-project knowledge and lifecycle records.
- Guaranteeing correctness of AI-generated content or independently certifying software quality.
- Autonomous production deployment by default; this may be integrated later under separately approved controls.
- A full learning management system, investment platform or public software marketplace in the initial commercial release.

1.4 Intended audience

The SRS is intended for the product owner, business analyst, software architect, engineering lead, backend/frontend developers, AI integration engineers, data engineer, security/privacy reviewer, QA team, DevOps/SRE team, tenant administrators and project stakeholders who approve scope and acceptance.

1.5 Product boundary

DevWeave is the system of record for project knowledge objects, approved revisions, relationships, activities, provenance, evidence metadata, stakeholder access policies, coordination state and external references. External code hosts remain authoritative for repository history and source code. External customer-managed agents or optional DevWeave-managed runners perform code changes in isolated workspaces; DevWeave records task contracts, runtime metadata, workspace references, commit/branch/PR references, outcomes and evidence. Source code may exist transiently inside an authorized managed execution workspace, but shall not be ingested into the canonical knowledge store, search/vector indexes, event payloads, analytics or durable audit records.

1.6 Requirement language and priority

The key words SHALL, SHALL NOT, SHOULD, SHOULD NOT and MAY are normative. Priority codes: M (Must), S (Should), C (Could). Release tags are MVP (technical foundation), R1 (first commercial pilot/launch), R2 (hybrid managed execution and commercial scale) and R3/FUT (enterprise and advanced future capabilities). MVP establishes the platform foundation; R1 is the first customer-facing product release and includes the agent-coordination wedge.

1.7 Standards and reference guidance

This SRS draws on ISO/IEC/IEEE 29148 and 15289 for requirements and lifecycle information, ISO/IEC 25010 for product quality, W3C PROV-O for provenance, JSON Schema for payload validation, NIST SP 800-162 for attribute-based authorization, and OWASP ASVS for application security verification. These references do not imply certification. Status shall be rechecked at each major baseline approval.

2. Overall Product Description

2.1 Product perspective

DevWeave is a hybrid AI software-delivery control plane composed of a canonical Knowledge Plane, a Coordination/Control Plane and one or more Execution Plane providers. Owner Studio, Agent Workspace, agency/client views, investor/reviewer views, learning/user experiences, administration and future applications shall consume the same canonical project information services. The Coordination Plane manages project-scoped MCP collaboration, execution campaigns, work-package assignments, runtime sessions, resource leases, usage/entitlement checks and integration queues. The Execution Plane may consist of customer-managed external runtimes, DevWeave-managed ephemeral runners, or both in one campaign. External repositories remain authoritative for source code.

2.2 Logical product layers

Layer	Responsibility	Examples
Experience	Stakeholder- and purpose-specific interfaces	Owner Studio, Agent Workspace, Investor Portal, Learning Portal, User Portal, Admin
Application/API	Stable APIs, auth, routing and validation	Versioned REST/OpenAPI plus a mandatory logical project-scoped MCP service for real-time agent coordination.
Domain	Canonical product capabilities	Tenancy, Projects, Knowledge, Relationships, Provenance, Lifecycle, Governance, Coordination, Agent Scheduling, Integration

Layer	Responsibility	Examples
Persistence/processing	Durable data, artifacts, search and dispatch	PostgreSQL, object storage, search index, background workers
External ecosystem	Specialized execution systems	Git hosts, coding agents, CI/CD, test systems, deployment platforms, identity providers
Coordination plane	Plan-locked distributed execution control; agent participation and assignment; work-package scheduling; leases; workspace isolation; integration/merge queue; campaign status	ExecutionCampaign, AgentRuntimeInstance, WorkPackage, TaskLease, WorkspaceReference, ChangeSetReference, IntegrationQueueItem

2.3 Core conceptual model

A project is an evolving information system, not a folder of files. Its canonical representation consists of KnowledgeItem identities, immutable KnowledgeRevision content, typed Relationships, Activities performed by Principals, Evidence and Approvals, lifecycle state and externally authoritative references.

```
Project Knowledge Model
  Project
    ├── KnowledgeItem ──< KnowledgeRevision
    ├── Relationship (source, type, target)
    ├── Activity ──< ActivityInput / ActivityOutput
    ├── Evidence ──< Verification / Review
    ├── LifecycleProfile / WorkflowInstance
    └── ExternalReference / ArtifactObject

Principal = Human | ServiceAccount | AI Agent
```

2.4 User classes and actors

Actor	Typical needs	Typical authority
Platform administrator	Operate platform, tenant lifecycle, application catalogue and platform config	Platform ops; no routine tenant-content access without approved grant
Tenant administrator	Manage members, apps, policies, quotas and tenant exports	Tenant-level administration
Project owner	Define scope, approve requirements/decisions, manage project	Project governance and configuration
Contributor	Create/update assigned knowledge, activities and artifacts	Scoped create/edit; approval may be separate
Reviewer/approver	Review proposals, evidence, changes and releases	Approve/reject within assigned scope
Investor/sponsor	Read approved progress, milestones, risk and evidence-backed reports	Read-only, purpose-limited access
Learner	Access approved explanations and selected history	Read-only curated access
Software user	Access released documentation, changes and known limitations	Public or account-scoped read/feedback
AI agent principal	Retrieve context, submit proposals, record execution and evidence	Machine identity with project/task scopes
External integration	Send/retrieve permitted metadata, events and references	Integration-specific scopes and credentials
Agent runtime instance / worker	Execute an assigned work package from a registered remote device, workstation, container or hosted runner; report heartbeat and results	Only the task, repository and resource scopes explicitly granted to that runtime session

Actor	Typical needs	Typical authority
Code-host integration / integration coordinator	Create or inspect branches/PRs/MRs, evaluate checks and coordinate merge queue actions	Provider token scopes and protected-branch policy; no source-code payload retained by DevWeave
Agent collaboration participant / MCP client	Discover authorized participants, exchange coordination messages, subscribe to campaign/channel changes, negotiate work and recover missed events	Project/campaign/channel scopes granted to the authenticated agent principal and runtime instance; no implied approval or repository authority
Economic buyer / workspace purchaser	Evaluate return on investment, subscription, managed execution consumption, security and procurement fit	Purchase/renew plan and approve commercial entitlements; no implicit project-content access
Agency delivery lead	Coordinate multiple client projects, agent capacity, delivery commitments, utilization and client reporting	Tenant/project-scoped operational authority as granted by the agency
Agency client / customer sponsor	Inspect agreed scope, milestones, delivery evidence, risks and approved reports for its project	Read-only or explicitly granted review/approval permissions limited to the contracted project
Managed execution administrator	Configure runner pools, regions, resource limits, images, credentials and runtime policies	Execution-plane administration scoped by tenant policy; no unrestricted knowledge/content access

2.5 Operating environment

- Web apps shall support current stable major evergreen desktop browsers; core reading/review flows shall be responsive on mobile.
- Backend services shall be deployable as containerized workloads in a cloud or customer-approved hosting environment.
- Initial reference stack: ASP.NET Core on a supported .NET LTS release, EF Core, PostgreSQL, S3-compatible object storage and Next.js/TypeScript. These are implementation baselines, not domain dependencies.
- Domain services shall remain protocol-neutral. Each project shall expose a logical project-scoped MCP coordination service as the standard real-time collaboration interface for MCP-capable agents. REST/OpenAPI and authenticated runtime adapters shall remain available for applications, administration, integrations and agents whose native runtime does not implement MCP; an adapter or local worker may bridge such agents to the project coordination service.
- The platform shall tolerate external-system outages and expose integration delivery status rather than silently dropping events.

2.6 Design and implementation constraints

- Each project shall have one owning tenant; cross-tenant participation shall be explicit, revocable and auditable.
- Tenant isolation shall be enforced at application, database and storage boundaries; application checks alone are insufficient.
- The primary data model shall work without a graph database; typed relationships may be relationally stored and indexed.
- Full event sourcing shall not be required for every aggregate; use current-state records, immutable knowledge revisions and append-only events where appropriate.
- Knowledge schemas and relationship types shall be registered and versioned; tenant extensions cannot redefine reserved platform types.
- AI-generated statements shall not automatically become approved or verified facts.
- No application interface shall bypass canonical domain authorization and validation.

2.7 Assumptions and dependencies

ID	Assumption / dependency	Impact if false
ASM-01	Initial service is hosted SaaS with pooled tenancy.	Dedicated database or single-tenant deployment needs a later profile.
ASM-02	Each project has one owning tenant; collaboration uses explicit grants.	True multi-owner projects require a different ownership/billing model.
ASM-03	External systems remain authoritative for source code, test/build, deployment and runtime.	First-party execution products expand scope materially.
ASM-04	Platform stores metadata, summaries, evidence and references from external systems.	Connector contracts and retention must be agreed per provider.
ASM-05	Supported object store, notification provider, identity provider and optional AI provider are available.	Some capabilities may need self-hosting or may be limited.

ID	Assumption / dependency	Impact if false
ASM-06	Product owner defines retention, privacy, residency and subscription policies before production.	Legal and operations gates remain open.
ASM-07	Initial target is modular monolith with bounded modules and workers.	Earlier service decomposition may be needed if scale/organization demands it.
ASM-08	Coordinated execution relies on supported external code-host APIs and reachable agent runtimes/workers; DevWeave is the control plane and does not host the source-code working tree.	Unsupported providers/runtimes require an adapter or are not eligible for coordinated campaigns.
ASM-09	The initial coordination model uses isolated workspaces/branches and one governed integration target per codebase campaign; direct multi-writer editing of one shared filesystem is not assumed.	Concurrent work without isolation would create race conditions, lost updates and unreproducible results.
ASM-10	A locked plan is immutable for the campaign baseline; plan changes use an explicit change-control and re-baselining process.	Uncontrolled plan drift would make agent assignments and acceptance results inconsistent.

3. Product Architecture and Design Constraints

3.1 Recommended implementation architecture

The initial implementation shall use a modular monolith with explicit module boundaries and independently scalable background workers. Knowledge, coordination, commercial entitlements and execution-provider integration shall be separately bounded domains. The architecture shall distinguish control-plane responsibilities from execution-plane responsibilities and permit later extraction of bounded modules/services without changing public domain identifiers or creating application-specific copies of project data.

Capability	Initial direction	Extensibility requirement
Identity/tenancy	OIDC-compatible identity; tenant/project memberships	Support additional identity providers and service principals
Canonical data	Relational records and JSONB payloads in PostgreSQL	Keep domain schema and APIs independent of one DB vendor
Relationships	Typed edge table with validated relation registry	Permit graph query service/projection later
History/provenance	Immutable revisions, activities and append-only events	Exportable provenance mappings and temporal reconstruction
Artifacts	Object storage with metadata/checksums/access mediation	S3-compatible provider abstraction and retention options
Search	PostgreSQL full-text initially	Pluggable search; vector search only when justified
Agent integration	REST/OpenAPI for protocol-neutral operations; project-scoped MCP is mandatory for real-time agent-to-agent coordination.	Keep domain operations protocol-neutral; expose coordination through the standard MCP service and allow additional protocols through adapters.
Applications	Shared APIs, separate application experiences	Application registry, scopes and versioned app contracts

3.2 Architectural principles

- Knowledge before documents: files are representations/payloads attached to typed knowledge identities.
- One canonical model: applications derive views from shared services and do not create conflicting copies.
- History before overwrite: important changes create new revisions or immutable events.
- Evidence before claims: reported completion, verification and approval remain distinct.

- Authorization before retrieval: permissions precede search, summaries, embeddings, analytics and AI context.
- Extensibility without semantic chaos: custom types/workflows require schemas, versions, ownership and validation.
- Hybrid execution boundary: external/customer-managed and optional DevWeave-managed runtimes perform code work in isolated workspaces; external code hosts remain authoritative, and the canonical knowledge store preserves metadata, references, outcomes and evidence rather than source-code payloads.
- Portable knowledge: exports include identifiers, schemas, revisions, relationships and provenance.

3.3 Technical baseline and version policy

At the SRS date (09 November 2026), .NET 10 is the recommended LTS baseline. Re-evaluate at implementation kickoff and each major release; use a vendor-supported release with current security patches. PostgreSQL 18 is the preferred initial database, JSON Schema Draft 2020-12 the default payload dialect, and OpenAPI 3.2.1 the preferred HTTP API description baseline where tooling is compatible. Pin versions in the implementation bill of materials and review at least quarterly.

The domain model shall not depend on .NET, PostgreSQL, JSON Schema, OpenAPI or MCP-specific types; adapters shall isolate these technology choices.

4. Stakeholders, Actors and Use Cases

4.1 Primary use cases

Use case	Name	Actor	Success outcome
UC-01	Create tenant and configure apps	Tenant admin	Tenant is provisioned, initial admin assigned, eligible apps enabled.
UC-02	Create/configure software project	Project owner	Project created, lifecycle selected, participants/references configured.
UC-03	Capture project knowledge	Owner/contributor/agent	Typed item is schema-validated, classified and related.
UC-04	Revise and approve knowledge	Contributor/reviewer	Revision proposed; authorized reviewer approves/rejects/requests change.
UC-05	Retrieve agent context	AI agent	Scoped agent receives authorized, relevant, revision-pinned information.
UC-06	Record agent execution	Agent/integration	Activity, inputs, outputs, refs, outcome and evidence are recorded without code.
UC-07	Verify requirement/release	Reviewer/QA	Evidence is assessed against criteria and result recorded.
UC-08	Generate stakeholder report	Owner/investor	Authorized information becomes evidence-linked metrics/narrative.
UC-09	Inspect history/impact	Owner/reviewer	Revision comparison and dependency impact are inspectable.
UC-10	Export project knowledge	Tenant admin/owner	Export includes schemas, revisions, relations, provenance and artifact refs.
UC-11	Integrate development tools	Tenant admin/integration	Connector is scoped, authenticated and configured for metadata events.
UC-12	Register extension type/profile	Tenant admin/curator	Extension validated/versioned/activated without mutating history.
UC-13	Lock plan and launch execution campaign	Project owner / approver	Approved planning snapshot, repository baseline and success criteria are frozen into a campaign baseline.

Use case	Name	Actor	Success outcome
UC-14	Register remote agent runtime and offer participation	Agent principal / runtime operator	A runtime registers capabilities, device/session state and availability; participation is subject to project authorization.
UC-15	Coordinate parallel work on one codebase	Coordination Plane / participating agents	Eligible agents receive non-conflicting work packages and work in isolated branches/workspaces against the same repository baseline.
UC-16	Integrate agent changes into the shared codebase	Integration coordinator / reviewer	Changes pass policy and required CI checks, are merged through a controlled queue, and the resulting commit is recorded.
UC-17	Recover interrupted distributed work	Coordination Plane / task owner	Expired sessions and leases are detected; work is safely checkpointed, resumed or reassigned without duplicate integration.
UC-18	Discover project agents and presence	Authorized agent / MCP client	The agent receives only permitted participant identity, capabilities and presence metadata for the current project or campaign.
UC-19	Coordinate with another agent in real time	Agent A and Agent B / Project MCP service	A durable message is committed, subscribed peers are notified, and recipients retrieve the ordered message using the channel cursor.
UC-20	Coordinate a group channel for a campaign	Campaign participants	Authorized agents join a project/campaign/work-package channel and exchange scoped coordination messages.
UC-21	Recover messages after disconnect	Reconnecting agent runtime	The runtime resumes from its last acknowledged cursor and obtains all authorized events/messages without relying on missed notifications.
UC-22	Negotiate handoff or request assistance	Participating agents / Coordination Plane	A proposed handoff is recorded, accepted or rejected explicitly, and task ownership changes only through the authoritative coordination/lease service.
UC-23	Configure a tenant plan and entitlements	Tenant admin / billing admin	Tenant receives a versioned plan with explicit project, concurrency, storage and execution entitlements.
UC-24	Connect customer-managed agent runtimes	Runtime operator / agent principal	External agent instances are enrolled, scoped and visible without requiring DevWeave-hosted compute.
UC-25	Launch a managed execution workspace	Project owner / runtime operator	An authorized task receives an isolated ephemeral managed workspace with approved credentials and limits.
UC-26	Run a hybrid campaign	Project owner / Coordination Plane	Customer-managed and DevWeave-managed agents contribute to one campaign under identical task, plan and integration rules.
UC-27	Review tenant usage and execution cost	Tenant admin / billing admin	Usage is visible by project, campaign, runtime class and billing period; metered records are auditable.
UC-28	Provide a client stakeholder view	Agency delivery lead / client sponsor	A client sees only the contracted project scope and approved progress/evidence information.

4.2 Representative agent-assisted delivery flow

1. Owner records approved goals, requirements, constraints, decisions and work items.
2. Owner assigns a work item to an agent principal with task-scoped permissions.
3. Agent requests context; platform captures a snapshot with exact revision references.

4. Agent works in an external coding environment; source code remains in the external repository. For coordinated campaigns, the platform dispatches plan-bound work packages to eligible agent runtime instances; each writer operates in an isolated workspace or branch against a recorded repository baseline.
5. Agent submits execution summary, proposals, repository/commit/PR refs, test-run refs and outcome. The Coordination Plane tracks lease ownership, dependencies, heartbeats, change-set references, integration gates and the campaign-level goal across all participating agents.
6. Platform validates submission, applies content policy and records provenance; unverified claims remain unverified.
7. Reviewer evaluates evidence and records approval, rejection or corrective action.
8. Stakeholder views refresh from canonical information and show evidence/freshness.

4.3 Stakeholder questions and data paths

Question	Required data path	Caveat
Owner: what remains before release?	Scope → work items → dependencies → verification → release criteria	Show blocked, unverified and disputed separately.
Investor: is milestone at risk?	Milestone baseline → activities → evidence → risks → approved changes	Show reporting date/scope/freshness; forecasts are not facts.
Agent: what context applies?	Task → requirements → decisions → contracts → constraints	Only authorized, pinned revisions.
Learner: why this architecture?	Decision → context → alternatives → rationale → consequences	Exclude restricted material; distinguish historical/current.
User: what changed?	Release → approved release notes → user-visible issues	Show released information only.

5. External Interface Requirements

5.1 User interfaces

- UI-01: Owner Studio shall provide project overview, settings, knowledge browser, relationship explorer, revision history, activity timeline, evidence review and access management.
- UI-02: Project-scoped screens shall identify the current tenant and project context.
- UI-03: The interface shall distinguish draft, proposed, approved, rejected, verified, failed, superseded and obsolete information.
- UI-04: Important displayed information shall show source, revision, last-updated time and verification status.
- UI-05: Core interfaces shall provide keyboard navigation, semantic labels, visible focus, text alternatives and responsive layouts.
- UI-06: Approval, revocation, export, deletion/retention change and integration authorization shall require explicit confirmation and audit.
- UI-07: Hiding a UI control shall not be treated as authorization; APIs shall enforce the same policies.

5.2 API interfaces

- API-01: The platform shall expose a versioned HTTPS API described by machine-readable OpenAPI.
- API-02: Responses shall use stable IDs, explicit schema versions, UTC timestamps and documented error semantics.
- API-03: List endpoints shall use bounded pagination and deterministic ordering; large traversals shall use cursor pagination.
- API-04: Mutating operations shall support idempotency keys where retries could create duplicates.
- API-05: Revision writes shall use optimistic concurrency (e.g., ETag/If-Match or expected revision).
- API-06: Tenant clients and agents shall not receive direct database or unrestricted query-language access.
- API-07: Backward-incompatible API changes shall require a new major version and published migration/deprecation policy.

5.3 Agent interfaces

- AGI-01: Agent access shall use a machine principal with revocable credentials and explicit scopes.
- AGI-02: Agent operations shall use protocol-neutral domain services. The platform shall provide a project-scoped MCP service as the standard and mandatory interface for real-time agent-to-agent coordination; REST/OpenAPI shall remain available for application, administration, integration and non-MCP use cases.
- AGI-03: Tool schemas shall describe validation, permissions, side effects, idempotency, errors and output references.
- AGI-04: Agents shall not receive unrestricted tenant search, approval authority or cross-project access by default.
- AGI-05: Context responses shall include item/revision IDs, scope, context snapshot ID and freshness metadata.

- AGI-06: Each project shall have a logically project-scoped MCP service identity, endpoint/resource namespace and policy configuration. Shared infrastructure may host many logical project services; a dedicated process per project is not required.
- AGI-07: The project MCP service shall expose authorized coordination tools and resources for agent discovery, presence, channels, messages, handoffs, coordination state and event replay.
- AGI-08: Authorized agent clients shall be able to subscribe to project/campaign/channel resource changes and retrieve durable messages/events by cursor after notification or reconnect.
- AGI-09: The platform shall mediate agent-to-agent collaboration. Agents shall not need direct network connectivity or direct trust relationships with one another.
- AGI-10: MCP transport notifications shall be treated as best-effort wake-up signals; durable message/event records and cursor-based replay shall be authoritative.

5.4 External system interfaces

Interface	Permitted data	Controls
Source-code host	Repository identity, default/protected branches, commit IDs, branch refs, PR/MR IDs, changed-path metadata where permitted, review state, check/run IDs and merge result	Allowlisted metadata only; external host is authoritative for source code; protected target branch; least-privilege tokens; merge queue and check policy
Coding agent/runtime	Agent identity, assigned work package, campaign and baseline IDs, context snapshot, lease token, workspace/branch refs, activity metadata, sanitized outcomes and evidence refs	Mutually authenticated transport; device/runtime identity; task-bound scope; heartbeat/lease; replay protection; source-code exclusion; no shared filesystem assumption
CI/test system	Pipeline/run IDs, outcomes, timestamps, test suite IDs, sanitized reports	Schema validation; secret/code scanning
Deployment platform	Release/deployment ID, environment, status, time, external URL, allowed health summary	Approval/environment scopes; no secret/config payload
Identity provider	Subject ID, auth result, group/claim attributes	OIDC/OAuth-compatible; no password storage under delegated auth
Object storage	Object key/version/checksum/size/media type/retention metadata	Private storage; short-lived access; tenant isolation
AI/model provider	Only approved context and output metadata	Provider allowlist, data terms, tenant opt-in, retention limits
Remote agent runtime / worker	Runtime instance ID, device/runner class, supported tools, capabilities, availability, region class (optional), workspace reference, heartbeat, execution result	Outbound authenticated connection preferred; short-lived credentials; device/session revocation; no implicit trust based on location; redact secrets/code
Workspace and integration adapter	Branch/worktree identifiers, base commit SHA, changed-path metadata, PR/MR reference, CI checks, merge queue status, merged commit SHA	Provider adapter; isolated workspaces; protected branches; serialized merge decision; store identifiers/metadata only, not code or diffs
Project-scoped MCP coordination service	Authorized agent discovery and presence; project/campaign/channel resources; agent messages; work proposals and handoffs; event cursors; coordination status; message and event acknowledgements	MCP protocol version negotiation; authenticated identity on every request; per-project/per-channel ACL; short-lived credentials; explicit handles; durable event log; best-effort notifications plus cursor replay; payload scanning; rate limits; audit

5.5 Notifications and webhooks

- WEB-01: Configured domain events shall be published to registered subscribers using authenticated delivery.
- WEB-02: Delivery semantics shall be documented; consumers shall deduplicate by event ID.
- WEB-03: Webhook requests shall use verifiable signatures or equivalent authentication with timestamp/replay protection.
- WEB-04: Failed deliveries shall use bounded backoff and an inspectable dead-letter state.
- WEB-05: Webhook payloads shall contain only fields authorized for the destination integration.

6. Functional Requirements

The following are externally observable product requirements. Each is intended to be atomic where practical. Acceptance text states the minimum verification condition; implementation design may add controls but shall not weaken the outcome.

6.1 Tenant, identity and membership

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
TEN-001	The system shall support multiple independent tenants within one SaaS platform.	M MVP	Create two tenants and verify independent IDs/settings.
TEN-002	Each tenant-owned project and knowledge record shall be associated with exactly one owning tenant.	M MVP	Database/API reject cross-tenant ownership references.
TEN-003	The system shall support human, service-account and AI-agent principals.	M MVP	Each principal has a stable ID and principal type.
TEN-004	The system shall support tenant membership with roles, status, invitation and revocation.	M MVP	Revoked member loses access on next authorized request.
TEN-005	The system shall support project-specific memberships and grants independent of tenant-wide roles.	M MVP	Tenant member without project grant cannot access restricted project.
TEN-006	The system shall permit explicit cross-tenant collaboration without changing project ownership.	S R2	Owner grant is scoped to named resources/actions and is revocable.
TEN-007	The system shall support OIDC-compatible federated authentication and secure service authentication.	M MVP	Success and failure flows tested with configured identity provider.
TEN-008	The system shall maintain tenant settings, plan/usage limits and application-installation state.	S R2	Settings are isolated; quota enforcement is observable.
TEN-009	Platform support access to tenant content shall require an approved, time-bounded, audited support grant.	M MVP	No standing support access exists by default.

6.2 Application catalogue and multi-application support

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
APP-001	The system shall maintain an application registry with stable IDs, versions, capabilities and lifecycle state.	S R2	Application metadata can be registered, queried and versioned.
APP-002	The system shall permit a tenant to enable or disable eligible applications independently.	S R2	Installation state controls access without duplicating project data.
APP-003	Applications shall consume canonical domain APIs and shall not maintain independent authoritative copies of project knowledge.	M MVP	Two application views return the same canonical revision ID.
APP-004	Future applications shall be addable without modifying existing knowledge identities or payloads.	M FUT	A test app consumes existing contracts without core schema rewrite.
APP-005	The system shall support application-specific navigation, dashboards and workflows while enforcing shared authorization.	S R2	Denied records remain inaccessible through all application routes.
APP-006	Application capabilities shall be declared and exposed through versioned contracts and permission scopes.	S FUT	App cannot invoke an undeclared capability.

6.3 Project management and lifecycle configuration

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
PRJ-001	An authorized tenant member shall be able to create any number of projects, subject to configurable platform/plan quotas.	M MVP	Create multiple projects; no fixed product-level count constant.
PRJ-002	Each project shall have stable identity, owning tenant, name, key, description, status and creation metadata.	M MVP	Required metadata persists and is returned consistently.
PRJ-003	The system shall support project membership, roles, scoped grants and revocation.	M MVP	A member accesses only assigned projects/actions.
PRJ-004	The system shall support draft, active, on-hold, completed and archived project states with configurable extensions.	M MVP	Transitions follow policy and are audited.
PRJ-005	A project shall reference multiple applications, services, repositories, databases and deployment environments.	M MVP	One project registers multiple typed external references.
PRJ-006	A project shall represent a product containing multiple deployable applications or services.	M MVP	One project contains multiple components without requiring one project per repository.
PRJ-007	The system shall support goals, scope, success criteria, assumptions, constraints and risks as typed items.	M MVP	Types validate and relate to requirements/milestones.
PRJ-008	The system should support project templates for types, lifecycle profiles, relationships and views.	S R2	Template creates configuration without copying private tenant data.
PRJ-009	The system shall support archive, restore, export and controlled deletion workflows.	M MVP	Actions are authorized, recorded and retention-aware.
PRJ-010	The system should support project custom attributes without changing reserved fields.	S R2	Custom values are schema-validated and cannot override reserved identifiers.

6.4 Universal knowledge object and schema registry

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
KNW-001	The system shall provide a universal KnowledgeItem identity independent of its content representation.	M MVP	Item retains same ID across multiple revisions.
KNW-002	Each KnowledgeItem shall have a registered type identifier and schema version.	M MVP	Creation fails for unknown or inactive type/version.
KNW-003	Each KnowledgeItem shall support common envelope metadata: tenant, project, key, title, classification, lifecycle, review, verification and currency state.	M MVP	Envelope fields are queryable as typed fields.
KNW-004	Typed payloads shall be validated against registered JSON schemas.	M MVP	Invalid payload rejected with path-specific validation errors.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
KNW-005	The system shall support text, structured data, diagrams, charts, documents, media and external-reference representations.	M MVP	Representative records in each initial family can be retrieved.
KNW-006	The system shall separate logical item identity, content revision, artifact binary identity and display representation.	M MVP	Payload update keeps item ID; file replacement creates artifact version.
KNW-007	The system should support tenant-defined extension types under platform governance.	S R2	Tenant type registration is namespace-scoped and validated.
KNW-008	The system shall expose type definitions and schemas to authorized clients and agents.	M MVP	Agent discovers supported type and schema versions.
KNW-009	The system shall support type deprecation/replacement without invalidating historical revisions.	M R2	Old types remain readable; new writes follow migration policy.
KNW-010	The system should support custom namespaced metadata fields without reserved-field collisions.	S R2	Metadata registration rejects reserved names.
KNW-011	The system shall distinguish proposed, approved, rejected and verified information states.	M MVP	Generated content cannot become approved merely by storage.
KNW-012	The system shall separate lifecycle, review, verification and currency dimensions.	M MVP	An item can be active, approved, unverified and current concurrently.
KNW-013	The system shall retain provenance metadata and source attribution on items and revisions.	M MVP	Originating actor/activity and source refs are retrievable.

6.5 Schema and extension governance

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
SCH-001	Each schema shall have a namespaced ID, version, owner, description and status.	M MVP	Registry rejects duplicate immutable versions.
SCH-002	The initial payload schema dialect shall support JSON Schema Draft 2020-12 or a documented compatible subset.	M MVP	Validation suite passes the supported schema vocabulary.
SCH-003	Published schema versions shall be immutable.	M MVP	In-place schema modification is rejected.
SCH-004	The system should assess schema compatibility before activating a new version.	S R2	Breaking changes flagged; migration/parallel version required.
SCH-005	The system should provide explicit, versioned payload transformations for migration.	S R2	Migration creates new revision and preserves original payload.
SCH-006	The system shall distinguish platform-reserved, platform-extension and tenant-defined namespaces.	M R2	Tenant registration cannot overwrite reserved definitions.
SCH-007	The system shall validate relationship endpoint types and cardinality against registry rules.	M MVP	Invalid typed relationship is rejected.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
SCH-008	The system should provide deprecation windows and compatibility documentation for consumed schemas.	S FUT	Consumers receive version/deprecation/migration metadata.

6.6 Relationships, graph traversal and traceability

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
REL-001	The system shall store typed directed relationships between KnowledgeItems.	M MVP	Source/target/type edge is created and retrieved.
REL-002	Each relation type shall define allowed source/target types, direction and cardinality.	M MVP	Registry validates permitted endpoint types.
REL-003	Initial relation types shall include contains, depends_on, satisfies, implements, validates, derived_from, generated_by, approved_by, supersedes, references and conflicts_with.	M MVP	Each relation type can be created and queried.
REL-004	The system shall support incoming/outgoing traversal with depth and result limits.	M MVP	Traversal returns correct paths and enforces bounds/permissions.
REL-005	The system shall support requirement-to-design-to-implementation-reference-to-test/evidence traceability.	M MVP	A test query returns an explainable trace path.
REL-006	The system should support impact analysis for changed or superseded items.	S R2	Change identifies related items under configured traversal rules.
REL-007	Invalid cross-project/tenant relationships shall be rejected unless an explicit supported grant permits the reference.	M MVP	Negative test denies unauthorized edge.
REL-008	Relationships should support provenance, effective dates, applicable confidence/strength and revocation state.	S R2	Metadata is preserved and queryable.
REL-009	The system should detect or explicitly permit cycles according to relationship-type policy.	S R2	Cycle validation follows declared rules.

6.7 Revisions, temporal history and events

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
REV-001	Substantive content changes shall create immutable KnowledgeRevision records.	M MVP	Prior revision remains retrievable after next revision.
REV-002	Each revision shall record revision number, schema version, actor, recorded time, reason and optional valid-time metadata.	M MVP	Required metadata exists for every revision.
REV-003	The system should compare revisions and show meaningful field-level changes where possible.	S R2	Structured payload comparison identifies changed paths.
REV-004	Revision creation shall support optimistic concurrency.	M MVP	Stale expected revision receives conflict response.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
REV-005	The system should preserve valid time and transaction/recorded time where temporal correction is required.	S R2	Late-recorded change queryable by both times.
REV-006	Security-relevant and domain-significant events shall be append-only to ordinary application roles.	M MVP	Ordinary roles cannot update/delete committed event records.
REV-007	The system shall reconstruct current or historical project state from retained revisions/events.	M MVP	Reconstruction test yields expected state at selected time.
REV-008	Corrections, retractions, supersession and approved redaction/deletion shall record reason and reference.	M MVP	Audit-visible tombstone/replacement reference remains.

6.8 Provenance, activity and evidence

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
PRO-001	The system shall model provenance using Entity, Activity and Agent/Principal concepts.	M MVP	Output revision links to an activity and actor.
PRO-002	An Activity shall record type, actor, times, status, purpose, inputs, outputs and correlation identifiers where available.	M MVP	Activity is queryable with input/output references.
PRO-003	The system shall record exact knowledge revisions used as activity inputs where known.	M MVP	Activity inputs are revision-pinned.
PRO-004	Generated outputs shall link to producing activity and responsible actor.	M MVP	Artifact has generated-by and actor association.
PRO-005	Claims, observations, evidence, verification results and approvals shall remain distinguishable.	M MVP	Claim without qualifying evidence remains unverified.
PRO-006	Evidence shall include origin, time, type, integrity metadata, related criterion and review state.	M MVP	Evidence record exposes required provenance/integrity fields.
PRO-007	The system should support evidence that contradicts or challenges an existing claim.	S R2	Contradictory evidence attaches without overwriting earlier evidence.
PRO-008	Human and machine activities shall use a common provenance model.	M MVP	Both principal types create compatible activity records.

6.9 Artifacts, documents and external references

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
ART-001	Permitted binary artifacts shall be stored in object storage with metadata in the canonical data store.	M MVP	Artifact metadata references object key and checksum.
ART-002	Artifact metadata shall include media type, size, checksum, tenant/project, creator, version and classification.	M MVP	Metadata is validated and queryable.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
ART-003	Artifact objects shall support immutable versions or equivalent version preservation.	M MVP	Replacement does not erase prior version outside retention rules.
ART-004	Artifact upload/download shall be access-mediated; permanent public URLs shall not be default.	M MVP	Unauthorized download denied; signed access expires.
ART-005	The system shall support external references to repositories, commits, PRs, pipelines, test runs, releases and deployments.	M MVP	External reference is typed, scoped and validated.
ART-006	Stored artifacts shall have integrity checksums calculated at ingest and revalidatable later.	M MVP	Checksum validation succeeds and tampering is detectable.
ART-007	The system should support retention, legal hold and controlled artifact deletion.	S R2	Retention/hold behavior demonstrated.
ART-008	Prohibited source-code content and secrets shall be rejected or quarantined before persistence/indexing.	M MVP	Test corpus covers UI, API, agent and connectors.
ART-009	The system should support preview/render metadata without requiring native editability for every format.	S R2	Supported formats preview; others retain download/reference metadata.

6.10 Lifecycle, work and governance workflows

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
LFC-001	The system shall support configurable lifecycle profiles rather than one mandatory methodology.	M MVP	A project can use the default profile and a second profile.
LFC-002	Lifecycle profiles should support stages, states, activities, transitions, conditions, dependencies and approval gates.	S R2	Configured profile validates allowed transition paths.
LFC-003	Lifecycle execution shall support parallel, iterative, branching, repeated, blocked, reopened and cancelled activities.	M MVP	Workflow test demonstrates non-linear execution.
LFC-004	The system shall support work items, milestones, dependencies, owners, due dates, status and priority.	M MVP	Milestone links to multiple work items and dependencies.
LFC-005	Planned state, reported execution state, verified state and approved state shall remain distinct.	M MVP	Dashboard does not equate agent-reported completion with verified completion.
LFC-006	Lifecycle transitions shall record actor, time and reason.	M MVP	Transition history includes required attributes.
LFC-007	Workflow configuration shall use declarative definitions and shall not execute arbitrary tenant-supplied code in the core process.	M R2	Schema validated; arbitrary script execution unavailable.
LFC-008	Lifecycle templates and project-specific overrides shall be versioned.	S R2	Template update does not silently rewrite active project instance.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
LFC-009	The system should support governance gates for requirements, architecture decisions, evidence acceptance and release approval.	M R2	Gate cannot pass without configured authority/evidence.

6.11 AI agent identity, context and execution

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
AGT-001	Each AI agent shall be represented as a machine principal with stable identity and lifecycle state.	M MVP	Agent can be created, disabled, rotated and audited.
AGT-002	Agent credentials shall be associated with tenant/project scopes, capability grants and expiry/revocation controls.	M MVP	Agent cannot access an ungranted project.
AGT-003	An authorized agent shall discover permitted knowledge types, relationship types and tool capabilities.	M MVP	Discovery output is filtered by capability/scope.
AGT-004	The system shall generate task-scoped context packages from authorized, relevant and revision-pinned knowledge.	M MVP	Context package includes item and revision references.
AGT-005	The system shall record a context snapshot ID and exact input revisions used by each agent execution.	M MVP	Execution traces to snapshot and revisions.
AGT-006	Agents shall submit proposed items/revisions without automatically approving them.	M MVP	Agent proposal enters proposed/pending-review state.
AGT-007	Agents shall record activities, inputs, outputs, status, errors and external references.	M MVP	Execution record is queryable and provenance-complete.
AGT-008	Beyond the mandatory project-scoped coordination service, the platform should expose suitable authorized knowledge-management and agent-domain operations through MCP tools where this improves interoperability.	S MVP	A compatible MCP client can invoke an authorized knowledge operation; coordination remains available through the project MCP service.
AGT-009	Equivalent core agent capabilities shall be available through protocol-neutral REST APIs.	M MVP	Core knowledge and agent operations remain available through REST/OpenAPI; real-time agent-to-agent coordination uses the mandatory project-scoped MCP service.
AGT-010	Agent submissions shall support idempotency and retry deduplication.	M MVP	Repeated idempotency key creates one logical result.
AGT-011	Stale agent revisions shall produce a conflict rather than silently overwrite newer knowledge.	M MVP	Stale write returns conflict and current revision reference.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
AGT-012	The system should capture provider/model/version metadata without requiring a particular vendor.	S R2	Execution has provider-neutral model metadata.
AGT-013	The system shall support human review, approval and revocation of agent capabilities.	M MVP	Capability change takes effect and is audited.
AGT-014	Agent/API errors shall be machine-readable and distinguish validation, authorization, conflict and transient failures.	S R2	Contract tests verify error categories.

6.12 Authorization, governance and audit

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
GOV-001	Tenant isolation shall apply to records, search indexes, artifacts, caches, events and background jobs.	M MVP	Cross-tenant tests fail closed across all paths.
GOV-002	Project-level authorization shall be enforced independently of tenant membership.	M MVP	Tenant member without project grant is denied.
GOV-003	The system shall combine roles with resource, subject, action and context attributes for authorization.	M MVP	Policy tests cover role/attribute combinations.
GOV-004	The system shall support resource classification and policy-based read/create/revise/approve/export/delete permissions.	M MVP	Restricted object denied to unauthorized role.
GOV-005	Authorization shall be applied before search delivery, AI summarization, vector retrieval, reports and exports.	M MVP	Indirect disclosure tests reveal no restricted facts.
GOV-006	Author/reviewer/approver privileges shall be separable where a workflow requires separation of duties.	M R2	Configured rule can prevent self-approval.
GOV-007	The system shall audit security-relevant access, grants, approvals, exports, integration changes and support access.	M MVP	Audit includes actor, action, target, time and outcome.
GOV-008	User, agent, integration and share grants shall be revocable.	M MVP	Revocation blocks subsequent access and invalidates/rotates tokens as applicable.
GOV-009	The system should support time-bounded external sharing and share revocation.	S R2	Expired grant no longer authorizes access.
GOV-010	The system shall support configurable retention, legal hold, export and controlled erasure by data category.	M R2	Policy is demonstrable in acceptance test.
GOV-011	Platform operators shall not routinely bypass tenant access controls.	M MVP	Support access requires approved time-limited grant and audit.
GOV-012	Tenant administrators should retrieve relevant tenant audit and security-administration history.	S R2	Authorized admin can retrieve tenant audit report.

6.13 Search, retrieval and context assembly

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
SRH-001	The system shall support full-text search across authorized metadata and permitted textual payloads.	M MVP	Search returns relevant authorized items.
SRH-002	Search shall filter by project, type, lifecycle, review, verification, currency, actor and date where relevant.	M MVP	Filters return correct subsets.
SRH-003	Results shall include item ID, title, type, current revision, status and provenance/freshness indicators.	M MVP	Result includes required metadata.
SRH-004	The system should support relationship-aware retrieval and bounded graph traversal.	S R2	Query retrieves related items within depth/result limits.
SRH-005	The system may support saved searches or reusable query definitions.	C FUT	Saved query can be reopened by authorized user.
SRH-006	The system may support semantic/vector retrieval over authorized text and metadata.	C FUT	Vector results enforce identical ACL and tenant scope.
SRH-007	The system shall expose indexing freshness and shall not imply new content is immediately searchable when indexing is asynchronous.	M R2	UI/API reports indexing state or last-indexed time.
SRH-008	The system should explain why an item was included in a context package or report.	S R2	Result includes inclusion reason/query or relationship trace.

6.14 Metrics, reporting and stakeholder views

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
RPT-001	The system shall provide a project overview of scope, lifecycle, milestones, activity, risks and evidence status.	M MVP	Owner dashboard aggregates canonical records.
RPT-002	The system shall distinguish task completion, implementation, verification, traceability and evidence-backed delivery metrics.	M MVP	Metrics are displayed separately with definitions.
RPT-003	Each calculated metric shall have a definition, scope, calculation version, timestamp and included/excluded population.	M MVP	Metric details are inspectable and reproducible.
RPT-004	Stakeholder views shall derive from canonical data under requester permissions and declared purpose.	M R2	Investor, owner and learner views show permitted shared records.
RPT-005	AI-generated factual report statements shall reference supporting items, revisions or evidence.	M R2	Report claim links to supporting records.
RPT-006	Estimates, predictions, unverified claims and stale evidence shall be labelled separately from verified facts.	M MVP	UI and exports preserve epistemic labels.
RPT-007	The system may support scheduled or event-triggered reports with delivery controls.	C FUT	Report is permission-checked at generation and delivery.

6.15 Integration and event processing

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
INT-001	The system shall provide an integration registry with provider, tenant, scope, credential reference and connection state.	M MVP	Configuration is tenant-scoped and inspectable.
INT-002	The system shall accept inbound events through authenticated APIs or webhooks.	M MVP	Valid event accepted; invalid signature or mapping rejected.
INT-003	The system shall deduplicate repeated external event IDs and process events idempotently.	M MVP	Duplicate delivery does not duplicate state changes.
INT-004	The system shall use durable dispatch (e.g., transactional outbox) for events published after a database transaction.	M MVP	Crash/retry test demonstrates no lost committed publication.
INT-005	The system should expose integration health, last sync, last error, retries and dead-letter state.	S R2	Administrator inspects connector state.
INT-006	Provider-specific identifiers shall be mapped through adapters, not used as core domain identifiers.	M R2	Two providers map to common external-reference contract.
INT-007	The system shall synchronize only approved metadata and shall not mirror source repositories.	M MVP	Connector allowlist contains metadata only.
INT-008	The system shall support disconnect, credential rotation and integration grant removal.	M MVP	Disconnected integration cannot access/write.

6.16 Export, portability and interoperability

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
EXP-001	The system shall export knowledge identities, revisions, relationships, schema versions, provenance and metadata.	M MVP	Export validates and re-imports into clean environment.
EXP-002	Export shall include permitted artifact binaries/references, checksums and classification metadata.	M MVP	Manifest maps artifacts to checksums and entries.
EXP-003	The system shall provide a documented native machine-readable export format.	M MVP	Format specification and sample export published.
EXP-004	The system should support JSON-LD/PROV-compatible mappings where applicable.	S FUT	Mapping preserves identity and provenance relationships.
EXP-005	Export shall preserve stable IDs and distinguish current from historical revisions.	M MVP	Import maintains IDs, revision order and current pointer.
EXP-006	The system shall audit export initiator, scope, destination mode and outcome.	M MVP	Export activity appears in audit history.
EXP-007	The system should provide documented exit/deletion workflows consistent with retention and legal hold.	S R2	Tenant-exit runbook and deletion evidence available.

6.17 Notifications and collaboration

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
COL-001	The system should notify authorized participants of configured review requests, approvals, rejections, assignments and integration failures.	S R2	Notification is sent only to eligible recipients.
COL-002	The system should support comments/annotations linked to an item/revision without altering the revision.	S R2	Comment separately attributed and auditable.
COL-003	The system should support assignment and escalation metadata for review/approval tasks.	S R2	Pending review assigned and visible to authorized user.
COL-004	The system may support user notification preferences and tenant notification policies.	C FUT	Preferences respected except mandatory security notices.

6.18 Platform administration and operations

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
OPS-001	The system shall provide platform administration for tenant lifecycle, application catalogue, operational status and platform configuration.	M MVP	Admin actions are scoped and audited.
OPS-002	The system shall expose health, readiness and dependency-health endpoints.	M MVP	Orchestrator detects unhealthy/unready instances.
OPS-003	The system shall emit structured logs, metrics and traces with correlation IDs.	M MVP	End-to-end request traceable without prohibited content.
OPS-004	The system shall support backup/restore for relational data, object metadata and binaries.	M MVP	Restore test meets approved RPO/RTO.
OPS-005	The system should support configurable quotas for storage, API, agents, projects and integration throughput.	S R2	Authorized administrator changes/enforces quota.
OPS-006	The system shall support controlled retention, purge and legal-hold execution with audit.	M R2	Retention job has preview, approval and audit result.
OPS-007	The system should support feature flags and progressive rollout for applications/schema capabilities.	S R2	Feature enabled for controlled tenant cohort.
OPS-008	Each release should include a software bill of materials and dependency/security update information.	S R2	Machine-readable SBOM included.

7. Information and Data Requirements

7.1 Canonical entities

Entity	Purpose / core attributes	Ownership and lifecycle
Tenant	Tenant ID, name, status, plan, region/policy settings	Platform governed; tenant admins maintain permitted settings
Principal	Principal ID, type (human/service/agent), identity provider reference, status	Global or tenant-scoped; credentials excluded from profile
Membership / Grant	Subject, tenant/project/resource, role/capabilities, validity, grantor	Revocable, time-bounded where suitable, audited
Project	Project ID, owning tenant, key, name, purpose, status, lifecycle profile	Exactly one owning tenant; state changes recorded
KnowledgeItem	Stable logical ID, project, type, current revision, classification, states	Stable identity; multiple immutable revisions
KnowledgeRevision	Revision ID, item ID, schema version, payload, actor, time, reason	Immutable except controlled redaction/tombstone process
Relationship	Source item, relation type, target item, metadata, provenance	Typed/validated; revoke or supersede without erasing history
Activity	Type, actor, times, purpose, inputs, outputs, status	Provenance record; external execution stays outside DevWeave
Evidence	Type, origin, reference/object, integrity, criterion, review state	May support or challenge claims; integrity metadata retained
ArtifactObject	Object key, media type, size, checksum, version, classification	Binary in object storage; access mediated by DevWeave
SchemaDefinition	Type ID, version, dialect, owner, schema, compatibility metadata	Published versions immutable and retained for history
ExternalReference	Provider, external ID, URI, resource type, sync status	Reference only unless approved metadata snapshot is stored
LifecycleProfile	Version, nodes, states, transitions, gates	Active project pins profile version or records migration
DomainEvent	Event ID, tenant/project, actor, type, aggregate, timestamp, correlation	Append-only within retention/immutability policy
PlanBaseline	Immutable snapshot of approved plan, goals, constraints, acceptance criteria, dependency graph and schema/revision references	Created by authorized lock action; immutable; superseded only through an approved new baseline.
ExecutionCampaign	A bounded coordinated implementation effort tied to one PlanBaseline and one or more CodebaseTargets	Tenant/project-owned; has objective, status, policy, participants, start/end and outcome.
CodebaseTarget	External repository identity, provider, target branch, initial commit SHA and branch protection/integration policy	Reference/metadata only; external code host remains authoritative.
AgentRuntimeInstance	A time-bounded execution instance of an Agent Principal on a device, runner or remote host	Registered, authenticated, capability-scoped, heartbeat-monitored and revocable.
ParticipationOffer	An agent/runtime offer to participate in a campaign with declared capability and availability	Must be authorized and accepted before assignment; offer is not an authorization grant.
WorkPackage	Plan-derived unit of implementation work with objective, scope, dependencies, constraints and acceptance criteria	Bound to campaign and plan baseline; versioned; may be split or replanned only through controlled change.
TaskLease	Time-bounded exclusive or scoped ownership of a work package/resource by a runtime instance	Renewable by heartbeat; expires/revokes safely; one active writer lease per exclusive resource.

Entity	Purpose / core attributes	Ownership and lifecycle
WorkspaceReference	External branch, worktree, environment or workspace identity associated with a work package/runtime	Reference and metadata only; workspace content remains external.
ChangeSetReference	External commit/branch/PR/MR and associated changed-path metadata for a submitted unit of work	Immutable external IDs captured; code/diff payload is not retained.
IntegrationQueueItem	Ordered candidate for integration with required policy checks, approvals and merge outcome	Only coordination service or authorized integration principal advances merge state.
CoordinationEvent	Agent joined/left, task claimed, lease renewed/expired, workspace created, check completed, merge accepted/rejected or campaign blocked	Append-only, correlated, tenant/project/campaign-scoped and idempotently ingested.
ResourceReservation	Time-bounded reservation of a logical resource such as a file-path scope, subsystem, API contract or database schema area	Conflict-aware; policy-scoped; does not imply a lock on the entire repository unless explicitly configured.

7.2 Data ownership and partitioning

- Every project shall have exactly one owning tenant; project identity is independent of display name or project key.
- Every tenant-owned data object shall carry tenant scope directly or through a required integrity-enforced parent.
- Cross-project references within a tenant shall be typed and policy-checked. Cross-tenant references require explicit owner-approved grants.
- External integration records shall be tenant-scoped; provider IDs shall not become primary keys of platform entities.
- Payloads shall be separate from common queryable envelope fields. Isolation, authorization, referential-integrity and core filtering fields shall not exist only in arbitrary JSON.

7.3 Revision and temporal rules

- Each KnowledgeItem shall have a stable identity and zero or more immutable revisions; a current-revision pointer identifies the selected revision.
- Revisions shall preserve the schema version under which the payload was validated.
- The system shall distinguish recorded/transaction time from valid/effective time where backdated corrections or imported history are supported.
- Corrections shall create a new revision or controlled redaction/tombstone event; silent overwrite is prohibited for governed content.
- Ordinary application roles cannot update/delete immutable revisions or events. Exceptional retention/legal deletion uses controlled privileged workflows with audit evidence.

7.4 Initial knowledge-object envelope

```
{
  "id": "uuid", "tenantId": "uuid", "projectId": "uuid",
  "typeId": "requirement.functional", "schemaVersion": "1.0.0",
  "key": "REQ-001", "title": "Support mobile payment",
  "classification": "confidential", "lifecycleState": "active",
  "reviewState": "approved", "verificationState": "unverified",
  "currencyState": "current", "currentRevisionId": "uuid",
  "createdBy": "principal-id", "createdAt": "2026-11-09T00:00:00Z"
}
```

Illustrative only; the final envelope is defined in the Knowledge Engine Domain Specification. The payload is validated using the registered type/schema. Fields used for isolation, access decisions, referential integrity and core filtering shall be first-class fields or trusted indexed metadata.

7.5 Initial payload example

```
{
 "description": "The product shall support an approved mobile payment gateway.",
 "priority": "high",
 "acceptanceCriteria": [
  "A user can initiate an authorized payment.",
  "The system records a payment outcome reference.",
  "The user receives a payment confirmation."
 ],
 "verificationMethod": "integration-test"
}
```

7.6 Data classification

Class	Definition	Default handling
Public	Approved for public release	Unauthenticated display only when publication is explicit.
Internal	Not public; intended for tenant/project participants	Authentication and authorization required.
Confidential	Sensitive business, project, architecture, investor or customer context	Need-to-know access; restricted sharing and export.
Restricted	Credentials, secrets, highly sensitive security/privacy material	Not ordinary knowledge; use an approved secret manager/restricted vault. Reject secret-bearing ingestion by default.

7.7 Source-code exclusion and content governance

Source-code exclusion is a product boundary, not merely a UI limitation. The system shall not intentionally ingest or retain source-code files, patch/diff content, executable source snippets, raw coding-session transcripts containing code, or source-code embeddings. Repository links, commit IDs, PR metadata, test-result identifiers, deployment references and high-level non-code summaries are permitted when authorized.

- All ingress paths (UI upload, API, agent tools, webhooks, connector sync, bulk import, search indexing and AI output capture) shall apply the same content policy.
- Controls shall use file-type rules, content inspection, secret scanning and quarantine/rejection. Detection is best-effort defense in depth, not a guarantee of perfect classification.
- Raw prompts, tool transcripts and execution logs shall not be retained by default. Only approved sanitized summaries and structured execution metadata may be retained.
- Embeddings, search indexes, caches and analytics shall not preserve prohibited code content indirectly.
- Uncertain potentially code-bearing content shall be quarantined or rejected pending authorized decision rather than silently stored.
- Error/audit logs shall not copy rejected source-code payloads.

7.8 Data integrity and uniqueness

Rule ID	Data rule
DATA-01	Project, KnowledgeItem, Revision, Activity, Event and Artifact IDs shall be globally unique or collision-resistant within a documented namespace.
DATA-02	Project-scoped keys shall be unique within their project; tenant-scoped keys shall be unique within their tenant where used.
DATA-03	Relationships shall reference existing source/target items in permitted scope.
DATA-04	Revision number shall be unique and monotonically increasing per KnowledgeItem.
DATA-05	Published schema version shall be immutable and identified by namespace plus version.

Rule ID	Data rule
DATA-06	Stored artifact shall have integrity checksum and immutable version identity.
DATA-07	External events shall be deduplicated by provider and external event identity where supplied.
DATA-08	Persist timestamps in UTC; display in user-configured locale/time zone.
DATA-09	Deletion/anonymization/retention shall preserve auditability to extent legally/contractually permitted.
DATA-10	Export shall preserve identity, revision order, schema reference, relationship direction and provenance.

8. Non-Functional Requirements

Quality requirements apply across all applications and tenants. Values marked “initial target” are provisional engineering baselines and must be validated with realistic workload tests before production SLA commitments.

ID	Quality requirement	Target / condition	Verification
NFR-001	Availability of core read/write APIs.	Initial target: 99.5% monthly, excluding approved maintenance and force majeure.	Monthly SLI report and synthetic monitoring.
NFR-002	Core metadata read latency under baseline load.	p95 ≤ 500 ms; p99 ≤ 1.5 s for common indexed reads.	Load test with agreed dataset/concurrency.
NFR-003	Knowledge revision write latency.	p95 ≤ 1.5 s excluding large binary upload/provider latency.	API performance test.
NFR-004	Full-text search latency.	p95 ≤ 3 s for agreed baseline project dataset.	Search benchmark/query tracing.
NFR-005	Search index freshness.	95% of eligible committed changes searchable within 30 s (initial target).	Measure commit-to-index delay.
NFR-006	Agent context package assembly.	p95 ≤ 5 s for up to 100 items or 5 MB text metadata; excludes model calls.	Context assembly performance test.
NFR-007	Initial service workload envelope.	Provisional: 500 concurrent interactive sessions and 100 sustained API requests/s.	Load and soak test; revise preproduction.
NFR-008	Horizontal scalability.	Stateless API/workers shall scale horizontally without tenant data redistribution as a prerequisite.	Scale-out test.
NFR-009	Backup recovery point objective.	Initial target RPO ≤ 1 hour for committed canonical data.	Restore and recovery exercise.
NFR-010	Disaster recovery time objective.	Initial target RTO ≤ 4 hours for core service restoration.	Timed recovery exercise.
NFR-011	Committed revision/event durability.	Successful commit survives ordinary process/node failure within configured DB durability guarantees.	Fault injection and restore test.
NFR-012	Tenant isolation.	No cross-tenant disclosure/modification via API, DB, storage, cache, search, logs or background work.	Isolation suite and penetration test.
NFR-013	Transport security.	Sensitive public/service traffic uses modern TLS; minimum baseline TLS 1.2+.	Configuration scan.

ID	Quality requirement	Target / condition	Verification
NFR-014	Encryption at rest.	Database backups, object storage and sensitive stores use managed encryption at rest.	Cloud and key-management audit.
NFR-015	Credential protection.	Passwords, API keys, OAuth secrets and agent credentials are never stored/logged in plaintext.	Secret scan and security review.
NFR-016	Authorization enforcement.	Every data path enforces tenant, project, classification and action policy server-side.	Authorization matrix testing.
NFR-017	Audit integrity.	Security/governance audit records are append-only to ordinary roles and protected from tampering.	Role and tampering tests.
NFR-018	Application security baseline.	Use OWASP ASVS 5.0.0; select and document target verification level/applicability.	ASVS-based assessment.
NFR-019	Vulnerability remediation.	Critical exploitable vulnerabilities block release; high-severity timelines defined in security policy.	Dependency/container/app scanning.
NFR-020	Privacy minimization.	Collect and retain only personal data required for stated functions/operations.	Privacy impact review and data inventory.
NFR-021	AI data-use controls.	No third-party model training on tenant data by default; external model sharing needs tenant policy approval.	Configuration/vendor processing audit.
NFR-022	Source-code exclusion.	No code-bearing content shall be intentionally stored, indexed, embedded or retained in logs.	Ingestion tests across all channels.
NFR-023	Portability.	Machine-readable export includes schemas, revisions, relations, provenance and permitted artifacts.	Export/import conformance test.
NFR-024	Interoperability.	Canonical API documented with OpenAPI; export and provenance mappings documented.	Contract tests and export validation.
NFR-025	Extensibility.	New types and application views addable without changing stable IDs or invalidating historical revisions.	Extension conformance test.
NFR-026	API backward compatibility.	Breaking changes within an API major version require a new version and published migration window.	API compatibility test.
NFR-027	Schema compatibility.	Published schema versions remain available to interpret retained historical revisions.	Schema registry/migration test.
NFR-028	Accessibility.	Core web experiences should conform to WCAG 2.2 AA, subject to third-party content limits.	Automated scan and manual audit.
NFR-029	Usability.	New owner can create project, add requirement and link architecture item without admin training.	Moderated usability test.
NFR-030	Observability.	Logs/metrics/traces correlate requests, jobs, events and agent runs without secrets or code.	Trace exercise and log scan.
NFR-031	Maintainability.	Domain modules have explicit contracts and automated tests; infrastructure adapters replaceable.	Architecture fitness tests/code review.

ID	Quality requirement	Target / condition	Verification
NFR-032	Deployability.	Services are containerizable and support repeatable automated deployment/rollback.	CI/CD rehearsal.
NFR-033	Migration safety.	Schema/data migrations are versioned, tested, reversible where feasible and backed up.	Production-like migration test.
NFR-034	Localization readiness.	User-facing text/date/number formatting externalized from business logic.	At least two-locale test.
NFR-035	Time handling.	Persist UTC; make valid-time and user-local display explicit.	Time-zone/DST test.
NFR-036	Failure transparency.	External failures, indexing delays and partial processing are shown, not silently reported as success.	Fault injection/UI/API test.
NFR-037	Retention configurability.	Retention configurable by data category and tenant policy, subject to platform minimums/legal holds.	Policy simulation and deletion audit.
NFR-038	Analytics isolation.	Aggregated metrics do not disclose identifiable tenant information to other tenants absent authorization.	Cross-tenant analytics test.
NFR-039	Capacity governance.	Quotas for storage, requests, agents, exports and integrations are observable/configurable.	Quota and burst-load test.
NFR-040	Dependency currency.	Production dependencies remain within vendor support and receive security updates under a policy.	SBOM and lifecycle review.

8.1 Performance measurement conditions

Performance shall be measured using a published workload profile specifying tenant/project counts, average items per project, payload sizes, relationship fan-out, artifact sizes, search selectivity, active users, agent executions and integration event rates. Tests shall include production-like authorization, audit, validation and indexing. A benchmark bypassing these paths is not valid acceptance evidence.

8.2 Scalability strategy

- Partitioning, read replicas, search extraction, object tiers, queue workers and service extraction may be introduced based on measured bottlenecks.
- No tenant shall access other tenants' workloads or indexes as a side effect of pooled infrastructure.
- Large tenants may later receive dedicated database/storage/search/compute through deployment profiles without changing the domain model.
- Graph traversal, report generation, export and context assembly shall have limits and asynchronous job options to prevent unbounded work.

9. Business Rules and Invariants

ID	Invariant / rule	Enforcement location
BR-001	Every project has exactly one owning tenant.	DB constraints + domain service
BR-002	A tenant may own multiple projects; no fixed product-level project limit.	Product model + quota service
BR-003	A project can contain multiple apps/services and link multiple repositories/environments.	Project domain
BR-004	Every knowledge item has stable identity and registered type.	Knowledge engine
BR-005	Every committed revision retains schema version used for validation.	Revision store + registry

ID	Invariant / rule	Enforcement location
BR-006	Published schema versions and committed revisions are immutable under ordinary operations.	DB permissions + domain service
BR-007	Every relationship conforms to registered relation contract.	Relationship service
BR-008	Cross-tenant access requires explicit owner-authorized, revocable grant.	Policy decision point
BR-009	Generated, reported, approved and verified are distinct states.	Domain model/UI/reporting
BR-010	A claim is not verified without qualifying evidence and accepted result.	Verification engine
BR-011	An AI agent is not an implicit superuser or tenant admin.	Identity/policy services
BR-012	Activity records exact input revisions where known/relevant.	Provenance service
BR-013	Stakeholder report cannot expose data requester cannot access directly.	Authorization/reporting
BR-014	External repositories remain authoritative for source code.	Integration policy
BR-015	Source code is not intentionally retained in DB, storage, search, embeddings, cache or logs.	Ingestion/observability
BR-016	Project lifecycle is configurable and may be non-linear.	Lifecycle engine
BR-017	Application view cannot update canonical facts outside shared domain API.	Application architecture
BR-018	Export preserves identity, schemas, revisions and relationship semantics.	Export service
BR-019	Retention/legal hold applies over ordinary deletion where legally applicable.	Retention service
BR-020	Metrics disclose estimates, stale/incomplete inputs or unverified basis.	Metrics/reporting

10. Security, Privacy and AI Governance

10.1 Security model

The platform shall use defense-in-depth authorization combining role-based access control (RBAC), resource/subject/action/context attributes (ABAC), and relationship-based project membership or sharing. Tenant membership alone shall not grant access to every project or object.

10.2 Mandatory security controls

- Authentication shall use a standards-based identity provider where practical. Service/agent credentials shall be scoped, rotated, revocable and stored in a protected credential store.
- Authorization shall be server-side and deny by default. Tenant context shall derive from verified identity and membership, not a client-supplied tenant ID.
- The relational store should use row-level security as defense in depth. App roles shall not have superuser/BYPASSRLS; table-owner bypass shall be considered and FORCE ROW LEVEL SECURITY applied where appropriate.
- Object storage shall use private buckets/containers, tenant-aware object keys, mediated authorization, short-lived signed access and encryption at rest.
- Caches, search documents, vector records, background jobs and event subscribers shall preserve tenant scope; sensitive delivery shall recheck authorization.
- State-changing security actions shall create audit records. Audit data shall not contain credentials, prohibited code or unnecessary personal data.

- Web application security verification shall use OWASP ASVS 5.0.0 as baseline; the security lead shall select target verification level and applicable controls.
- Production secrets shall be managed separately from application configuration and shall not be committed or embedded in images.

10.3 Privacy and tenant data control

- The product shall maintain a data inventory with personal-data categories, purpose, source, storage location, recipients, retention and deletion method.
- Tenants shall be able to configure permitted retention and external AI processing policies within platform/legal constraints.
- Tenant content shall not be used to train a third-party model by default. Opt-in must be explicit, informed, tenant-scoped and recorded.
- External AI use shall disclose provider, data categories sent, purpose, processing/retention terms where available and tenant controls.
- The system shall support export, correction, deletion/anonymization and legal hold consistent with applicable law and contract. This SRS does not determine jurisdiction-specific obligations.
- Data residency, cross-border transfer, subprocessors and processing terms shall be decided before production onboarding of regulated/enterprise tenants.

10.4 AI-specific governance

Control	Requirement
Agent identity	Every agent activity attributable to a registered machine principal and, where applicable, initiating human/workflow.
Least privilege	Agent permissions limited by tenant, project, task, tool, action and duration.
Context control	Context packages authorized, revision-pinned, purpose-scoped and recorded.
Output status	Agent output enters proposed/reported state unless a narrow policy explicitly authorizes automated acceptance.
Human oversight	High-impact approvals, capability grants, release gates and production-impacting actions require configured human authorization.
Prompt/data hygiene	Secrets, code and restricted content excluded/redacted before external model processing.
Model metadata	Provider/model/version recorded where available; no model vendor is mandatory.
Auditability	Tool invocation identity, time, scope, outcome and linked inputs/outputs recorded without prohibited raw code transcripts.
Plan binding	Every coordinated work package, runtime assignment and integration candidate is bound to one immutable plan baseline and campaign.
Workspace isolation	Concurrent writers use isolated workspaces/branches or explicitly partitioned non-overlapping scopes; no uncoordinated shared working directory.
Lease discipline	Task/resource ownership is time-bounded, heartbeat-renewed, revocable and safely expired; stale sessions cannot continue to hold authority.
Protected integration	Agents do not bypass protected target branches or the integration queue; merge is subject to branch policy, checks and approval rules.
Remote runtime trust	A device/location is not trusted solely by network location; runtime identity, posture, scope and session credentials are verified.
Code boundary	The platform records code-host identifiers and permitted coordination metadata only; source files, diffs and raw code-bearing session data remain external.

10.5 Threats requiring explicit testing

- Cross-tenant identifier guessing and insecure direct object reference.
- Privilege escalation through tenant roles, project grants, agent credentials or background job context.
- Indirect leakage through search, AI summaries, embeddings, exports, notifications or analytics.

- Prompt injection/malicious project content causing agent to request unauthorized data or tools.
- Replay, duplicate delivery, forged webhook, compromised integration credential and stale agent token.
- Untrusted file parsing, malicious document payloads, archive bombs, malware and content scanner evasion.
- Abuse of schema registration, relationships, workflow configuration or report queries to cause resource exhaustion.
- Accidental source code/secret retention in logs, caches, model context, embeddings or external payloads.

11. Verification and Acceptance

11.1 Verification methods

Method	Use	Evidence
Inspection	Review schema, configuration, policy, UI, documentation and architecture boundary	Review checklist and signed result
Analysis	Evaluate formulas, invariants, access policy, capacity and traceability	Analysis report/reproducible query
Demonstration	Show end-to-end user or agent scenario	Recorded steps and observed result
Test	Functional, integration, security, performance and recovery execution	Test case, run ID, result and evidence
Audit	Assess governance, retention, provenance, access and change control	Audit report and remediation log

11.2 MVP acceptance gates

Gate	Acceptance condition
AC-01 Tenant isolation	Two or more tenants exist; tests show no cross-tenant access through APIs, DB, artifacts, search, cache, events, exports or background jobs.
AC-02 Multi-project	A tenant creates multiple projects; each has separate memberships, knowledge, settings and external references.
AC-03 Knowledge lifecycle	Users create typed objects, validate payloads, revise, retrieve prior revisions and compare revisions.
AC-04 Typed relations	Users connect goals, requirements, decisions, work items, tests and evidence; invalid relationships are rejected.
AC-05 Provenance	A knowledge output is traceable to activity, actor and exact input revisions.
AC-06 Agent context	Scoped agent retrieves a revision-pinned context package; unauthorized/restricted items are absent.
AC-07 Agent execution	Agent records inputs, outputs, outcome and permitted refs; retry does not duplicate activity.
AC-08 Verification integrity	Agent-reported completion does not count as verified delivery without policy-required evidence review.
AC-09 Code exclusion	Source-code files, diffs and code samples are rejected/quarantined across UI/API/agent/import/connectors; prohibited payload absent from persisted stores/logs/indexes.
AC-10 Artifact management	Permitted document/diagram uploaded, checksummed, versioned and permission-checked.
AC-11 Owner Studio	Owner inspects project state, knowledge, relations, history, activities and evidence.
AC-12 Export	Export preserves IDs, schemas, revisions, relationships, provenance and artifact manifest and validates.
AC-13 Operations	Health checks, structured logs, metrics, traces, backup/restore and rollback demonstrated.
AC-14 Security review	Agreed ASVS-based verification has no unresolved release-blocking findings.

Gate	Acceptance condition
AC-15 Plan lock and campaign baseline	An authorized owner locks an approved plan; the system creates an immutable baseline containing approved revisions, dependencies, constraints and success criteria. A later change requires an approved change request/new baseline.
AC-16 Distributed agent participation	At least two authorized agent runtime instances on separate devices or network locations join one campaign, are capability-checked and receive scoped work packages tied to the same plan baseline and codebase target.
AC-17 Parallel isolated work	Two or more agents execute independent/partitioned work packages against the same external repository from recorded base commits using isolated branches/workspaces; no shared-write race is required.
AC-18 Conflict and lease safety	A conflicting task/resource claim is rejected or serialized; an expired/offline runtime loses its lease; duplicate/replayed commands do not create duplicate work or merges.
AC-19 Controlled integration	A change set is linked to its external PR/MR/commit, required checks and approval; the protected integration branch is updated only through the configured merge queue/policy.
AC-20 End-to-end goal traceability	The campaign view links the locked goal and plan baseline to work packages, runtime instances, activity, change-set references, merged commits, tests and final acceptance evidence.
AC-21 Remote interruption and recovery	Disconnect one runtime during execution; heartbeat timeout, lease expiry, checkpoint/reassignment and recovery are recorded without false completion or silent loss of authoritative status.
AC-22 Code-host boundary	Demonstrate full coordinated flow while verifying DevWeave stores repository/branch/commit/PR/check metadata but no source files, patch bodies or code diffs.
AC-23 Hybrid control/execution separation	A campaign can use customer-managed runtimes while the control plane stores only authorized metadata/references; execution provider and workspace identity are explicit.
AC-24 Customer-managed remote agents	At least two customer-managed agent runtimes on separate devices/networks join one campaign, exchange real-time MCP coordination events and execute isolated work packages.
AC-25 Optional managed execution	An authorized task can be launched in an isolated managed workspace with scoped credentials, resource limits, auditable lifecycle and cleanup; this gate is R2.
AC-26 Mixed-mode campaign	One campaign can assign one work package to an external runtime and another to a DevWeave-managed runtime without changing the plan baseline or governance rules; this gate is R2.
AC-27 Entitlement enforcement	A tenant plan controls configured limits such as active projects, concurrent agent sessions, managed execution allowance, storage and retention; denied/over-limit actions return actionable responses.
AC-28 Usage metering and transparency	Usage events are idempotent, attributable to tenant/project/campaign/runtime class and reconcilable to a usage statement; model-provider spend paid directly by the customer is not billed by DevWeave unless a separate resale contract exists.
AC-29 Agency/client access boundary	An agency client can access only explicitly shared project reports and evidence; no other client project or tenant data is disclosed.

11.3 Test levels

- Unit tests for domain rules, schema validation, relationship cardinality, revision conflicts, policy evaluation and metric calculations.
- Integration tests for PostgreSQL constraints/RLS, object storage, identity, transactional outbox, webhook signatures, search and connectors.
- Contract tests for OpenAPI, agent tool schemas, integration payloads, schema registry and export format.
- End-to-end tests for owner workflow, agent context/execution, stakeholder reporting, approval gates and tenant export/exit.
- Negative security tests for tenant leakage, unauthorized traversal, prompt-injection-driven tool abuse, artifact URL leakage and report disclosure.
- Performance, soak, fault-injection, backup/restore and disaster-recovery tests under a documented workload profile.

11.4 Requirement traceability

Every requirement shall map to at least one design element and verification artifact before release. Acceptance shall not be based only on code completion or UI demonstration. The traceability record shall include requirement ID/version, design/API/schema reference, implementation work item, test case/version, test result, release version and any waiver.

12. Deployment, Operations and Support

12.1 Deployment model

- Initial deployment comprises containerized web/API instances, background workers, PostgreSQL, private object storage and optional search/cache services.
- Production, staging, test and development environments shall be isolated. Production data shall not be copied to lower environments without masking and access approval.
- Infrastructure shall be provisioned reproducibly through infrastructure-as-code or an equivalent documented process.
- Database migrations shall use controlled deployment steps, compatibility checks and backups. Destructive changes should use expand-migrate-contract or equivalent safe rollout.
- Application configuration and secrets shall be environment-specific and separate from container images.

12.2 Monitoring and operations

Area	Minimum requirement
Availability	Synthetic checks for authentication, project read, knowledge read/write and artifact access.
Performance	API/database latency, queue depth, search latency, context assembly and export duration.
Data integrity	Revision/event write failures, outbox lag, checksum failures, schema errors and relationship violations.
Security	Auth failures, denied access, grant changes, secret alerts, integration credential changes and unusual exports.
Agent operations	Execution count/failure, context size, tool error types, duplicate submissions and review backlog.
Integrations	Connector health, webhook delivery/retries, last sync and dead-letter queue.
Cost/usage	Storage, egress, AI usage, index volume, compute/queue and tenant quota consumption.

12.3 Backup and recovery

The production runbook shall define backup scope/frequency for relational data, object metadata/binaries, schema registry, integration configuration and encryption-key recovery. Initial planning targets are RPO ≤ 1 hour and RTO ≤ 4 hours, subject to design, budget and measured restore exercises. A backup is not valid acceptance evidence until restoration is tested.

12.4 Support and tenant exit

- Support staff shall use approved time-limited grants to inspect tenant data; access shall identify reason/ticket and be recorded.
- Tenants shall be able to request machine-readable export of their data and permitted binaries.
- Tenant closure shall disable access, revoke credentials/integrations, apply retention/legal holds and delete after approved retention period.
- Before general availability, platform support shall publish status, incident communications and security contact procedures.

13. Release Scope and Delivery Strategy

13.1 MVP baseline

The MVP proves the knowledge model, project boundary, provenance, revision safety, tenant isolation and agent integration. It does not attempt to deliver every stakeholder application or extension capability.

Included in Foundation MVP	Deferred to R1/R2/R3
Tenant/project management, memberships, pooled tenant isolation	Dedicated tenant deployment and enterprise federation
Universal knowledge items and initial type catalogue	Broad schema marketplace and custom type designer
JSON Schema validation, immutable revisions, relationship registry	Advanced migration assistant and graph analytics

Included in Foundation MVP	Deferred to R1/R2/R3
Activities/provenance, evidence metadata, basic review	Advanced workflow designer and compliance packs
Document/diagram storage with checksums and access controls	Rich native editor for every format
Agent principal, REST APIs, context packages and execution records; basic agent connector contract and project-level MCP identity/configuration foundations.	R1 adds sellable plan-locked multi-agent coordination: project MCP real-time collaboration, external runtime registration, work packages, leases, isolated workspaces, integration queue and campaign evidence. R2 adds optional DevWeave-managed execution, mixed-mode campaigns, usage metering/billing automation and agency/client portal capabilities. R3 adds advanced enterprise deployment and intelligence.
Full-text search, Owner Studio, basic project dashboard	Advanced vector/hybrid search and stakeholder app builder
Source-code exclusion controls at ingress/retrieval	Guarantee of perfect content classification
Native project export, audit, backup/health checks	JSON-LD interchange suite and dedicated tenant infrastructure

13.2 R1 Commercial Pilot, R2 Hybrid Growth and Future Roadmap

Horizon	Capabilities
R1 — Commercial Pilot	Plan lock and immutable campaign baseline; one initial code-host integration with provider-neutral adapter contract; customer-managed distributed agent runtimes; project-scoped MCP real-time messaging, discovery, presence, channels and durable replay; work packages and dependency graph; leases and isolated workspaces; controlled PR/MR integration; basic subscription entitlements and usage visibility; agency-focused Owner/Delivery Studio.
R2 — Hybrid Growth	Optional DevWeave-managed ephemeral agent execution; mixed customer-managed/managed campaigns; managed compute provisioning and usage metering; billing-provider integration; advanced capability-based scheduling; multiple code-host and agent-provider adapters; agency client portal; expanded analytics, notifications, retention and admin controls.
R3 / FUT — Enterprise	Private-cloud/customer-hosted deployment profiles; enterprise identity federation and advanced governance; data residency and policy packs; advanced semantic retrieval, impact analysis, portfolio intelligence, extensible application catalogue and specialized compliance capabilities.
Long-term optional	Cross-organization collaboration marketplace, specialized compliance modules, richer agent protocols, customer-hosted deployment options and domain ontology packs.

13.3 Release exit criteria

- All M/MVP foundation requirements and all M/R1 requirements for the targeted commercial pilot have passing evidence or approved waiver with owner, rationale, risk and expiration. R2/R3 requirements are gated by their respective release plans.
- No unresolved critical security finding; high-risk findings have explicit release decision and remediation plan.
- Tenant isolation and source-code exclusion suites pass on every ingress and retrieval path.
- Backup/restore and rollback exercises pass in production-like environment.
- Operations runbooks, monitoring, support escalation and export documentation are approved.
- Data model, schemas, API/event contracts and export format are versioned and published.

14. Traceability and Requirement Governance

14.1 Requirement attributes

The engineering requirement register shall maintain at least: ID, title, statement, rationale, priority, release, owner, source, dependencies, acceptance criteria, verification method, design reference, implementation work item, test case, status, version and change history.

14.2 Change control

- Every requirement change shall include reason, affected stakeholders, proposed wording, priority/release impact, data migration impact, security/privacy impact and compatibility assessment.
- Changes to tenant isolation, code exclusion, provenance semantics, immutable history, approval authority or exportability are architecture-significant and require product, architecture and security review.
- New knowledge/relationship types shall be reviewed for semantic overlap, schema stability, classification, retention, export mapping and search behavior.
- Approved changes shall update this SRS or a controlled supplement and update traceability links.

14.3 Traceability from product principles

Principle	Requirement families	Acceptance gates
P-01 Knowledge before documents	KNW, SCH, REL, ART, SRH	AC-03, AC-04, AC-10
P-02 History before current state	REV, PRO, EXP	AC-03, AC-05, AC-12
P-03 Evidence before claims	PRO, LFC, RPT	AC-08, AC-11
P-04 One canonical model, many views	APP, RPT, GOV	AC-01, AC-11
P-05 Extensibility without chaos	SCH, REL, LFC, APP, EXP	AC-03, AC-04, AC-12
P-06 Tenant isolation by design	TEN, GOV, NFR	AC-01
P-07 Source code stays external	ART, AGT, INT, BR-015	AC-09
P-08 Agent actions attributable	AGT, PRO, REV	AC-05, AC-06, AC-07

14.4 Release traceability rule

Every release requirement shall trace to implementation and tests. Test results shall identify exact requirement and test versions. A test passed against an earlier requirement revision shall not automatically verify changed wording.

15. Assumptions, Dependencies, Risks and Open Decisions

15.1 Material risks

Risk	Description	Severity	Mitigation
R-01	Knowledge model becomes generic and loses domain meaning.	High	Reserved types, semantic relations, validation and governance.
R-02	Tenant customization fragments schemas and reporting.	High	Namespaces, schema review, compatibility, common envelope and export tests.
R-03	Code leaks into logs, embeddings or agent transcripts.	High	Allowlist ingest, inspection, quarantine, transcript minimization and E2E tests.
R-04	Search/report creates indirect cross-tenant disclosure.	Critical	Authorize before retrieval/summarization; isolate indexes; negative disclosure tests.
R-05	Agent claims are mistaken for verified facts.	High	Separate epistemic states, evidence rules, approval gates and clear UI.

Risk	Description	Severity	Mitigation
R-06	History grows without cost/retention strategy.	Medium	Retention tiers, archive, partitioning, legal holds and quotas.
R-07	Premature microservices increase complexity.	Medium	Start modular monolith; extract measured bounded contexts only.
R-08	External APIs change or fail.	Medium	Provider adapters, retries, idempotency, health and external-reference model.
R-09	AI provider terms conflict with confidentiality.	High	Provider allowlist, tenant opt-in, processing review and alternative path.
R-10	Progress metrics incomparable across methodologies.	High	Metric registry, explicit populations and per-project profiles.
R-11	Large traversals/reports exhaust resources.	Medium	Depth/result caps, query budgets, async jobs and rate limits.
R-12	Old schemas become unreadable.	High	Immutable registry, compatibility tests, export validation and long-lived readers.
R-13	Multiple agents edit overlapping resources or integrate incompatible changes.	Critical	Default to isolated workspaces, resource leases, conflict detection, protected branches and serialized integration queue.
R-14	Remote runtime disconnects while holding work or a lease.	High	Heartbeat expiry, lease revocation, checkpoint metadata, safe reassignment and idempotent submissions.
R-15	A locked plan changes while agents execute against the prior baseline.	High	Immutable campaign baseline, change request, impact analysis and controlled re-baselining/migration.
R-16	Untrusted device/runtime obtains code-host or project permissions beyond its assignment.	Critical	Short-lived credentials, least privilege, runtime attestation/policy, revocation and external branch protections.
R-17	Platform attempts to host or copy code in order to coordinate agents.	High	Control-plane boundary; use external workspaces and metadata-only repository integration.
R-18	Agent-to-agent notifications may be lost during network interruption or client restarts.	High	Persist messages/events before acknowledging; use MCP subscriptions as wake-up signals; provide per-channel sequence numbers, cursors, acknowledgements and replay; test reconnect and duplicate handling.
R-19	Agents may receive misleading, malicious or unauthorized messages from other participants.	High	Authenticate every sender; enforce channel membership and project policy; treat messages as untrusted input; prohibit implicit tool execution and privilege inheritance; audit and provide moderation/revocation.
R-20	Different MCP client implementations may not support long-lived resource subscriptions consistently.	Medium	Publish a protocol conformance profile; provide reconnectable subscriptions and cursor-based read/poll fallback; validate client capabilities and maintain adapter compatibility.

15.2 Open decisions before production baseline

ID	Decision	Question	Owner	Due
OD-01	Commercial tenancy/quotas	Plans, storage, agent/project limits, overage and billing.	Product/finance	Commercial launch

ID	Decision	Question	Owner	Due
OD-02	Data residency/legal terms	Regions, subprocessors, cross-border transfers, DPA and jurisdiction.	Legal/security	External tenants
OD-03	Retention schedule	Retention by content, activity, audit, artifact, backup and legal hold.	Product/legal/ops	Production
OD-04	AI provider policy	Allowed providers, retention, opt-in, self-hosting and tenant selection.	Product/security	External AI use
OD-05	Dedicated tenant option	Need for dedicated DB/storage isolation.	Product/architecture	R2 planning
OD-06	Public user documentation	Public vs authenticated docs and release binding.	Product owner	R2 planning
OD-07	Diagram/document editing	Native editing vs preview/storage and integrations.	Product/UX	MVP UX freeze
OD-08	Evidence assurance levels	Evidence types and human approvals per project profile.	Engineering/QA	Release gates
OD-09	Application extension model	First-party-only vs third-party app/plugin marketplace.	Product/security	Before marketplace
OD-10	Final SLO/workload profile	Concurrency, throughput, growth, RPO/RTO, availability and SLA.	Engineering/ops	Production commitment
OD-11	Initial code-host providers	Which providers are supported for branch, PR/MR, CI status and merge queue automation in R2?	Product/engineering	R2 design freeze
OD-12	Remote runtime protocol	Outbound WebSocket, long-polling, worker SDK or a combination; how are local and hosted runners enrolled?	Platform/agent engineering	R2 design freeze
OD-13	Workspace provider model	Which external workspace/branch/worktree provisioning adapters are supported first?	Engineering/DevEx	R2 design freeze
OD-14	Concurrency policy	Default lease durations, path/subsystem reservation policy, paired-agent mode and merge queue rules.	Engineering/QA	R2 design freeze
OD-15	Plan-lock authority	Which roles may lock a plan, approve re-baselining and authorize production-impacting integration?	Product owner/security	Before R2 pilot
OD-16	Agent trust posture	What device posture/attestation evidence is required for local laptops versus managed runners?	Security/operations	Before remote-runtime launch
OD-17	MCP protocol compatibility	Which minimum client protocol versions and compatibility adapters must be supported during the first coordinated-agent release?	Platform/agent engineering	R2 design freeze
OD-18	Coordination channel policy	Which channel types (project, campaign, work-package, team and direct agent-to-agent) are enabled by default, and what are their retention and moderation rules?	Product/security	R2 design freeze

ID	Decision	Question	Owner	Due
OD-19	Real-time workload profile	What number of concurrent runtimes, subscribed channels, message rate and geographic latency profile will be used to confirm production SLOs?	Engineering/operations	Before production SLA
OD-20	Commercial plan catalogue	Which plan tiers, entitlements, included concurrency, project/storage limits and trial/free policy will apply at launch? Prices must be held outside the SRS in a versioned commercial catalogue.	Product/finance	Before R1 pricing
OD-21	Usage meter definitions	Which usage dimensions are billable: concurrent sessions, managed runtime minutes/compute units, storage, retention, active campaigns or support tier? Which are informational only?	Product/finance/engineering	R1/R2 pricing
OD-22	Managed execution provider model	Which regions, runner images, compute providers, workspace persistence and cleanup policies will be supported for the first managed execution beta?	Engineering/security/finance	R2 design freeze
OD-23	Billing and tax provider	Which payment/invoicing provider, currency, tax handling, credit/refund and failed-payment lifecycle will be supported for self-service and enterprise contracts?	Finance/legal	R2 billing launch
OD-24	Agency client access model	Will clients be invited as read-only stakeholders, reviewers or approvers, and which project/report objects may be shared?	Product/security	R2 agency portal
OD-25	BYOK and provider credential policy	Which model/provider credentials may be supplied by tenants, how are they isolated, rotated, metered and excluded from DevWeave billing?	Product/security	R1/R2
OD-26	Enterprise deployment options	Which single-tenant, private-cloud or customer-hosted profiles will be supported and priced?	Product/architecture/finance	R3 planning

Baseline recommendation

Do not block domain modelling on all commercial questions. Resolve schema, tenancy, provenance, authorization and source-code boundary first. Defer billing, advanced stakeholder portals and optional deployment profiles until the MVP proves the canonical knowledge model.

Appendix A. Initial Knowledge Type Catalogue

Starter catalogue, not a closed list. Each type shall have a schema, description, owner, lifecycle policy, default classification, allowed relationships and migration strategy.

Type	Purpose	Illustrative payload fields
strategy.goal	Business/product objective	statement, rationale, successMeasures, owner, targetDate
strategy.assumption	Assumption requiring validation	statement, source, confidence, validationOwner, dueDate
requirements.functional	Observable functional behavior	description, priority, rationale, acceptanceCriteria, source

Type	Purpose	Illustrative payload fields
requirements.nonfunctional	Quality/operational requirement	qualityAttribute, metric, threshold, condition, verificationMethod
requirements.constraint	Design/legal/technical/business constraint	constraintType, statement, rationale, scope
architecture.decision	Architecture decision and rationale	context, decision, alternatives, consequences, status
architecture.component	Logical/deployable component	name, responsibility, interfaces, dependencies, technologyReference
architecture.interface	Contract between components/systems	protocol, operations, schemaReference, version, owner
specification.technical	Detailed technical specification	purpose, scope, normativeContent, references
planning.work-item	Planned unit of work	objective, owner, status, priority, estimate, dueDate, dependencies
execution.activity-report	Human/agent work summary	activity, outcome, inputs, outputs, errors, externalRefs
verification.test-case	Verification procedure/case	objective, preconditions, stepsReference, expectedResult, requirementRefs
verification.result	Test/assessment result	runRef, status, observedResult, evidenceRefs, environment
governance.risk	Uncertainty with impact	description, likelihood, impact, mitigation, owner, status
delivery.release	Release candidate or released state	version, scope, approvalRefs, releaseDate, notesRef
delivery.deployment	External deployment record	releaseRef, environment, providerRef, status, timestamp
operation.incident	Operational event/incident	severity, summary, affectedService, impact, resolutionRef
learning.lesson	Educational explanation/retrospective	topic, audience, explanation, learningObjectives, sourceRefs
experience.feedback	User/stakeholder feedback	source, audience, category, summary, disposition, relatedItemRefs

Appendix B. Initial Relationship Type Catalogue

Relationship	Source → target	Meaning / validation
contains	Parent → child	Containment; cycle rules depend on hierarchy.
depends_on	Consumer → dependency	Consumer requires target; may be transitive.
satisfies	Design/specification → requirement	Target requirement is addressed by source.
implements	Implementation reference → requirement/specification	External implementation is claimed to realize target.
validates	Test/evidence → requirement/criterion	Evidence supports verification of target.
derived_from	New item → source item/revision	Derivation/provenance.
generated_by	Artifact/revision → activity	Output was generated by activity.
used_by	Input item/revision → activity	Activity consumed the input.
associated_with	Activity → principal	Principal participated in/performed activity.
approved_by	Item/revision → principal/approval	Approval decision by authorized actor.
supersedes	New item/revision → old item/revision	New state replaces old while preserving history.

Relationship	Source → target	Meaning / validation
conflicts_with	Item ↔ item	Claims/artifacts are inconsistent or contested.
references	Item → external/internal resource	Non-semantic reference; does not imply validation.
deployed_as	Release → deployment	Release is associated with deployment record.

Appendix C. Canonical Data Concepts and Examples

C.1 Activity provenance example

```
Approved Requirements v3
  └── used by → Activity: Generate Architecture
                ├── associated with → Architecture Agent
                ├── generated → Architecture Document v1
                └── generated → Architecture Diagram v1

Architecture Document v1
  └── reviewed by → Human Reviewer
      └── Approval: Approved
```

C.2 Agent context package example

```
{
 "contextSnapshotId": "uuid", "projectId": "uuid",
 "taskId": "TASK-125", "createdAt": "2026-11-09T00:00:00Z",
 "items": [
  {"itemId":"REQ-001","revisionId":"uuid","reason":"Approved requirement"},
  {"itemId":"ADR-004","revisionId":"uuid","reason":"Applicable decision"},
  {"itemId":"API-003","revisionId":"uuid","reason":"Relevant interface contract"}
 ],
 "constraints": ["Do not change public contract without approval"],
 "scope": {"tenantId":"uuid","projectId":"uuid","taskId":"TASK-125"}
}
```

Illustrative only. Actual packages are generated after authorization filtering and shall not expose hidden records or internal policy secrets.

C.3 Suggested native export bundle

```
project-export/
 manifest.json
 tenant/tenant.json
 project/project.json
 schemas/schema-index.json
 schemas/<type-id>/<schema-version>.json
 knowledge/items.jsonl
 knowledge/revisions.jsonl
 knowledge/relationships.jsonl
 provenance/activities.jsonl
 provenance/events.jsonl
 provenance/evidence.jsonl
 provenance/approvals.jsonl
 artifacts/artifact-manifest.json
 artifacts/objects/<content-hash>
 external-references/references.jsonl
 checksums.sha256
```

Appendix D. Glossary

Term	Definition
Activity	Recorded action performed by a human, agent or external system.
Agent principal	Machine identity representing an AI agent or autonomous software actor.
Artifact	File, diagram, chart, report, image or other retained representation attached to knowledge.
Canonical model	Authoritative shared domain representation used by all applications.
Claim	Statement about the project that may or may not be verified.
Context package	Permission-filtered, purpose-specific, revision-pinned knowledge set supplied to an agent or stakeholder.
Evidence	Record/reference supporting or challenging a claim, requirement, test, approval or outcome.
KnowledgeItem	Stable logical identity for a typed piece of project knowledge.
KnowledgeRevision	Immutable version of content associated with a KnowledgeItem.
Lifecycle profile	Versioned configuration of stages, states, activities, transitions and approval gates.
Provenance	Information about origin, derivation, use, generation and responsibility.
Project	Software initiative that may contain multiple applications, services, repositories and environments.

Term	Definition
Tenant	Independently governed SaaS customer/account boundary that owns projects.
Valid time	Period during which a fact is considered true in the modeled domain.
Transaction/recorded time	Time at which the platform recorded a fact or change.
Verification	Documented assessment of a stated criterion against evidence.
Source-code exclusion	Rule preventing source-code content from being intentionally retained in DevWeave.

Appendix E. References and Standards

References inform this SRS. Standards may change; status shall be reviewed when establishing contractual, compliance or major-release baselines. Accessed 09 November 2026.

[R1] ISO (2018). ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering. https://www.iso.org/standard/72089.html *Published edition; ISO lists a replacement draft under development in 2026.*

[R2] ISO (2019). ISO/IEC/IEEE 15289:2019, Systems and software engineering — Content of life-cycle information items. https://www.iso.org/standard/74909.html *Published edition; successor DIS registered in October 2026.*

[R3] ISO (2023). ISO/IEC 25010:2023, Systems and software engineering — SQuaRE — Product quality model. https://www.iso.org/standard/78176.html *Quality model reference.*

[R4] W3C (2013). PROV-O: The PROV Ontology. https://www.w3.org/TR/prov-o/ *W3C Recommendation for provenance representation.*

[R5] JSON Schema (2022). JSON Schema Draft 2020-12. https://json-schema.org/draft/2020-12/ *Current published schema dialect identified by project site at SRS date.*

[R6] NIST (2019). NIST SP 800-162, Guide to Attribute Based Access Control. https://csrc.nist.gov/pubs/sp/800/162/upd2/final *ABAC guidance for attributes of subject, object, action and environment.*

[R7] OWASP (2025). OWASP Application Security Verification Standard 5.0.0. https://asvs.owasp.org/ *Security verification baseline; team selects applicable level.*

[R8] PostgreSQL Global Development Group (2026). PostgreSQL 18 Documentation: Row Security Policies. https://www.postgresql.org/docs/18/ddl-rowsecurity.html *RLS behavior; table-owner and privileged-role bypass must be addressed.*

[R9] OpenAPI Initiative (2026). OpenAPI Specification 3.2.1. https://spec.openapis.org/oas/v3.2.1.html *Published 10 September 2026.*

[R10] .NET / Microsoft (2026). Official .NET support policy. https://dotnet.microsoft.com/en-us/platform/support/policy *Use supported releases and revalidate LTS at kickoff/major releases.*

Implementation handoff

Next controlled engineering artifacts: (1) Knowledge Engine Domain Specification, (2) logical/physical database model, (3) JSON Schema catalogue, (4) OpenAPI and agent-tool contracts, (5) security threat model and authorization matrix, and (6) verification/traceability plan.

16. Amendment 1.1 — Multi-Agent Distributed Execution and Shared-Codebase Coordination

This section is a normative coordination specification within the DevWeave SRS. It makes plan-locked multi-agent coordination over shared external codebases a first commercial-release capability (R1). It shall be read together with Sections 1–15 and the business-aligned release strategy in Section 18. Where an earlier statement or release table places core project MCP coordination and customer-managed distributed execution in R2, Section 18 and this release alignment take precedence. Optional DevWeave-managed execution is an R2 capability. Permanent source-code hosting or repository replacement remains out of scope.

16.1 Purpose and scope

The platform shall coordinate multiple authorized AI agents, potentially operating from different physical locations, devices, operating systems, containers or hosted runners, to implement one approved plan against the same authoritative external codebase or defined set of codebases. The agents shall work toward one campaign objective, share a consistent plan baseline and coordinate dependencies, work ownership, resource conflicts, workspace isolation, change submission, verification and integration.

The platform is the coordination control plane, not the source-code data plane. Source code remains in the external repository and execution workspaces. The platform stores task instructions and approved non-code context, repository and commit identifiers, branch/workspace references, lease state, changed-path metadata where permitted, pull/merge-request references, CI/test identifiers, status, sanitized summaries and evidence metadata. It shall not store source files, patch bodies, diffs, executable code snippets or raw code-bearing transcripts.

“Same codebase” means that participating work packages target the same registered authoritative repository identity and campaign integration target (or an explicitly declared set of repositories). It does not mean that agents are permitted to write concurrently into one shared local working directory. Concurrent implementation shall use isolated workspaces/branches or explicitly partitioned scopes, followed by controlled integration.

16.2 Terminology and conceptual entities

Term	Definition
Plan lock	An authorized governance action that freezes a specific approved set of plan/requirement/decision revisions and dependency relationships as an immutable execution baseline.
PlanBaseline	A versioned, immutable snapshot of the approved scope, objective, constraints, success criteria, applicable lifecycle/profile, work decomposition and referenced knowledge revisions.
ExecutionCampaign	A bounded coordination instance created from one PlanBaseline and one or more CodebaseTargets to deliver a common goal.
CodebaseTarget	A reference to an external repository, provider, canonical repository ID, target branch, initial commit SHA and integration policy.
Agent Principal	The durable identity of an AI agent or agent service. It is distinct from a running process or device.
AgentRuntimeInstance	A specific authenticated process/session of an agent running on a laptop, workstation, container, server or hosted runner. Multiple instances may exist for one Agent Principal subject to policy.
ParticipationOffer	A proposal by an agent/runtime to participate in a campaign, including capabilities, availability and requested scope. It does not itself grant authorization.
WorkPackage	A plan-derived implementation unit with objective, allowed scope, constraints, dependencies, acceptance criteria, required capabilities and integration expectations.
TaskLease	A time-bounded grant assigning a WorkPackage or protected logical resource to one runtime instance or explicitly defined collaboration group.
WorkspaceReference	Metadata reference to an external branch, worktree, container/workspace or development environment used for a work package.
ChangeSetReference	A reference to a commit, branch, PR/MR or provider change set, including external identifiers and permitted metadata but not code/diff contents.

Term	Definition
IntegrationQueueItem	A controlled candidate for integration into the campaign target branch after policy checks and approvals.
ResourceReservation	A time-bounded reservation for a logical resource scope such as a path group, subsystem, interface contract or database schema area.

16.3 Coordination architecture and responsibilities

The Coordination Plane shall be a bounded domain module in the initial modular monolith. It may later be extracted into independently scalable services without changing its public domain contracts. It shall include the following logical capabilities:

- Execution Campaign Manager: creates campaigns from locked plan baselines, binds common goals and repository targets, and manages campaign lifecycle.
- Agent Registry and Runtime Session Manager: registers agent principals and runtime instances, records declared capabilities and availability, authenticates sessions, tracks heartbeats and supports revocation.
- Participation and Scheduling Service: receives agent offers, evaluates eligibility and capability fit, and dispatches authorized work packages.
- Work Package and Dependency Manager: converts or maps approved plans into implementable work packages, maintains dependency DAGs, scope boundaries and acceptance criteria.
- Lease and Resource Coordination Service: grants, renews, expires and revokes task/resource leases; prevents conflicting active writers for exclusive scopes.
- Workspace/Repository Adapter: requests or records isolated branches/workspaces through external providers and binds them to base commits and campaign targets.
- Integration Coordinator: evaluates change-set references, CI/test signals, approvals and merge policies; serializes integration into protected target branches.
- Campaign Telemetry and Recovery: aggregates status, failures, disconnects, blockers, reassignments and goal-level evidence; supports pause/resume/recovery.

The platform shall not require all agents to run on platform-hosted compute. Remote agent runtimes shall connect through authenticated outbound connections or supported provider APIs. The platform shall not assume shared local storage, stable IP addresses, a single operating system or a permanently connected device.

16.4 Operating model and lifecycle

The standard coordinated execution flow shall be:

1. Complete planning: the project owner and authorized reviewers approve goals, requirements, architecture, constraints, work dependencies, acceptance criteria and applicable policies.
2. Lock plan: an authorized actor locks the plan. The platform creates an immutable PlanBaseline referencing exact KnowledgeItem revisions, relationship versions, lifecycle profile version and success criteria.
3. Register codebase targets: the owner links the campaign to the external repository or repositories, protected target branch and starting commit SHA. The integration policy is recorded.
4. Create campaign: the platform creates an ExecutionCampaign bound to the PlanBaseline and CodebaseTargets. No work package may execute against an unbound or ambiguous baseline.
5. Mobilize agents: authorized agents/runtimes discover eligible campaigns or receive invitations, register capabilities/availability and submit ParticipationOffers. The owner or policy engine authorizes participation.
6. Prepare work packages: plan items are decomposed or mapped into work packages with explicit scope, objective, dependencies, allowed resources/paths, constraints and acceptance criteria.
7. Assign and lease: the scheduler assigns eligible work packages; the Lease Service issues short-lived task/resource leases and a context snapshot pinned to the plan baseline.
8. Execute remotely: each writing runtime uses an isolated workspace/branch based on the recorded repository commit. Agents report start, heartbeat, progress, checkpoints, blockers and results.
9. Submit change sets: an agent submits external commit/branch/PR/MR references and permitted metadata. The platform records provenance and links outputs to the work package and campaign.
10. Verify and integrate: configured checks, review and approvals run. Eligible changes enter a serialized integration queue and are merged into the authoritative target branch only through policy.
11. Update campaign state: the merged commit SHA, test evidence and accepted work-package state update the campaign projection. The overall goal is complete only when campaign acceptance criteria are met.
12. Close or evolve: campaign completion, cancellation or supersession is recorded. A changed plan is handled through change control and a new baseline or explicitly approved migration of the campaign.

16.5 Functional requirements

The following requirements are mandatory for the first commercial coordinated-execution capability. Priority “M / R1” means mandatory for the R1 commercial pilot/launch; the technical foundation MVP may precede this release.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
MCO-001	The system shall provide a Coordination Plane as a first-class domain capability separate from the Knowledge Engine and external code execution runtime.	M / R1	Architecture and module-boundary inspection shows independent coordination contracts and persisted campaign state.
MCO-002	The system shall permit an authorized owner to lock an approved project plan and create an immutable PlanBaseline.	M / R1	Lock action records actor, time, plan revision set, relationship/dependency snapshot and baseline ID.
MCO-003	A PlanBaseline shall identify the common goal, approved scope, constraints, acceptance criteria, dependencies and exact source revisions used for execution.	M / R1	Baseline export/retrieval reproduces the same identifiers and revision references.
MCO-004	The system shall reject campaign execution if required planning approvals, baseline data or repository targets are incomplete.	M / R1	Campaign cannot transition to Mobilizing/Executing until configured gates pass.
MCO-005	The system shall create an ExecutionCampaign bound to exactly one active PlanBaseline version and one or more declared CodebaseTargets.	M / R1	Campaign record references baseline and repository targets; invalid/missing binding is rejected.
MCO-006	The system shall preserve the locked baseline unchanged while agents execute; modifications shall be submitted as change requests.	M / R1	Attempted mutation is rejected; change request is separately recorded.
MCO-007	The system shall support approval, impact analysis and controlled re-baselining when a plan change affects an active campaign.	M / R1	Approved change creates a new baseline/version and records affected packages, sessions and decisions.
MCO-008	The system shall register one or more external repositories/codebases per project and bind campaign targets to canonical repository identities.	M / R1	A campaign supports a repository target with provider ID, repository ID, URL, branch and base commit reference.
MCO-009	The system shall record the initial base commit SHA and target integration branch for each CodebaseTarget.	M / R1	Each work package/workspace records the baseline commit used.
MCO-010	The system shall support provider adapters for repository, branch, PR/MR, status-check and merge metadata without becoming a source-code host.	M / R1	Adapter contract tests retrieve and persist permitted metadata only.
MCO-011	The system shall distinguish an Agent Principal from one or more AgentRuntimeInstances operating from different devices, locations or execution environments.	M / R1	One principal may have multiple independently auditable runtime instances.
MCO-012	Each AgentRuntimeInstance shall register a stable session ID, agent principal, runtime/device class, declared capabilities, software/protocol version and lifecycle state.	M / R1	Runtime registration validates required attributes and produces a revocable session.
MCO-013	The system shall record optional coarse execution region/location metadata only when needed for policy, scheduling or residency; precise personal location shall not be required.	S / R2	Runtime can operate without precise location; policy can use configured region class.
MCO-014	The system shall support authenticated remote runtime connection from different networks/devices without requiring inbound access to a participant device.	M / R1	A worker behind NAT/firewall can establish an authorized outbound session or provider-mediated connection.
MCO-015	The system shall monitor runtime heartbeat, session health and last-seen time and shall detect stale/disconnected instances.	M / R1	Heartbeat timeout transitions the session to stale/offline and emits a coordination event.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
MCO-016	An agent/runtime shall be able to discover or be invited to campaigns and submit a ParticipationOffer containing capabilities, availability and requested scope.	M / R1	Offer is recorded but no work authorization is granted until accepted.
MCO-017	The system shall authorize campaign participation separately from agent registration and shall evaluate tenant, project, task, tool, repository and time scope.	M / R1	Registered but unauthorized agent cannot retrieve campaign data or receive a lease.
MCO-018	The scheduler shall match work packages to eligible runtime instances using declared capabilities, authorization, availability, resource limits and policy constraints.	M / R1	Ineligible runtime is excluded and scheduling decision is auditable.
MCO-019	The system shall support owner-directed, policy-directed and capability-based work-package assignment.	S / R2	Campaign can use at least one configured assignment policy and record the decision basis.
MCO-020	The system shall map approved plan elements into WorkPackages with objective, scope, constraints, dependencies, acceptance criteria and required capabilities.	M / R1	Every executable package has a complete validated work contract.
MCO-021	The system shall represent work-package dependencies as a directed acyclic graph unless an explicitly modelled iterative loop is supported by the workflow profile.	M / R1	Dependency validation rejects accidental cycles and reports blocked descendants.
MCO-022	Each WorkPackage shall reference the campaign, PlanBaseline, relevant knowledge revisions, CodebaseTarget, target commit and expected outputs.	M / R1	Work package retrieval includes all baseline and repository references.
MCO-023	The system shall support task states including Ready, Offered, Assigned, Leased, InProgress, Submitted, ChecksPending, ReadyForIntegration, Integrated, Blocked, Rework and Cancelled.	M / R1	Invalid transitions are rejected and permitted transitions are audited.
MCO-024	The system shall issue a time-bounded TaskLease before an agent may perform authorized write work for a package or exclusive resource.	M / R1	Write-capable task requires a valid lease token and scope.
MCO-025	By default, an exclusive WorkPackage or exclusive resource shall have no more than one active write lease at a time.	M / R1	Concurrent conflicting claim is denied or queued; lease history is retained.
MCO-026	The system shall support explicit concurrency modes: exclusive-writer, partitioned-writers and review-only; any shared-session mode shall identify one authorized writer/coordinator.	M / R1	Mode controls number and scope of writer leases; invalid overlap is rejected.
MCO-027	The system shall support ResourceReservations for conflict-prone logical scopes such as path groups, components, API contracts or database schema areas.	S / R2	Overlapping exclusive reservations cannot be granted concurrently.
MCO-028	The system shall renew leases only from authenticated active runtime sessions and shall expire/revoke leases after configured heartbeat/lease policy is breached.	M / R1	A stale or revoked session cannot renew or use the lease.
MCO-029	The system shall support safe checkpoint, pause, resume and reassignment of work after runtime disconnect, cancellation or failure.	M / R1	Recovery preserves previous execution evidence and prevents stale lease holders from submitting as current owner.
MCO-030	The system shall create or record an isolated branch/worktree/workspace reference for each concurrent writing assignment unless a defined provider-specific isolation strategy is approved.	M / R1	Two agents can work concurrently in distinct workspaces against the same repository and recorded base commit.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
MCO-031	The system shall bind each workspace to its runtime, work package, campaign, codebase target, base commit and lease.	M / R1	Workspace provenance is complete and queryable.
MCO-032	The system shall prevent agents from directly bypassing protected integration branches when campaign policy requires pull/merge request or merge-queue integration.	M / R1	Unauthorized direct push/merge is rejected by platform credentials and/or repository protection policy.
MCO-033	The system shall accept ChangeSetReferences for external commits, branches, PRs/MRs and changed-path metadata without ingesting code or diff content.	M / R1	Reference is stored and code/diff body is absent from platform stores and indexes.
MCO-034	The system shall detect stale base commits, overlapping changed-path metadata and declared resource conflicts before integration.	M / R1	Stale/conflicting candidate is flagged or blocked according to policy.
MCO-035	The system shall provide a serialized IntegrationQueue for eligible changes targeting the same protected branch, preserving queue order and revalidating checks where required.	M / R1	Two change sets cannot race an uncontrolled merge into the same protected target.
MCO-036	The system shall evaluate required CI/test/status checks, review decisions and approvals before marking a change eligible for integration.	M / R1	Missing/failed required check blocks integration and explains the reason.
MCO-037	The system shall record the resulting merged commit SHA, external merge result, check identifiers and associated accepted work packages.	M / R1	Merged work package is traceable to repository, branch, commit and evidence.
MCO-038	The system shall support rejection, rework, revert-reference recording and rollback status without rewriting prior coordination history.	M / R1	Rejected/reverted change leaves an immutable history and updates current campaign projection.
MCO-039	All agent tasks, activities, outputs, change sets and integration decisions in a campaign shall be linked to the same campaign goal and PlanBaseline unless explicitly migrated.	M / R1	Campaign traceability query shows common goal/baseline for all participating work.
MCO-040	The system shall provide a campaign dashboard showing participating agents/runtimes, online state, assignments, leases, work-package dependencies, blockers, change sets, queue state, merged commits and acceptance evidence.	M / R1	Dashboard values reconcile to canonical coordination records.
MCO-041	The system shall emit coordination events for offer, join, assignment, lease issue/renew/expire, heartbeat timeout, workspace creation, progress, block, submission, check, queue and integration transitions.	M / R1	Events are correlated to campaign, project, actor, runtime and work package where applicable.
MCO-042	The system shall use idempotency keys and unique external event identifiers to prevent duplicate assignments, submissions and merge-result processing.	M / R1	Repeated provider webhook/request yields one logical state transition.
MCO-043	The system shall allow authorized owners to pause, resume, cancel or terminate a campaign and revoke affected runtime sessions/leases.	M / R1	Control action takes effect within the defined revocation target and is audited.
MCO-044	The system shall preserve campaign-level goal completion criteria separately from agent-reported task completion and shall require configured verification evidence before campaign completion.	M / R1	Campaign cannot complete merely because every agent reports success.
MCO-045	The system shall support multiple campaigns over the same project/repository while detecting target-branch or resource conflicts according to repository policy.	S / R3	Conflicting campaigns are serialized, isolated or blocked per configured policy.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
MCO-046	The system shall expose protocol-neutral coordination APIs and may expose equivalent authorized operations through MCP or other agent protocols.	M / R1	A non-MCP client can complete registration, claim, heartbeat, submit and status workflows.
MCO-047	The system shall support configurable participation policy, including allowlist, owner invitation, capability threshold, trust posture and manual approval.	S / R2	Campaign policy determines eligibility and produces an auditable decision.
MCO-048	The system shall provide a kill switch to suspend a runtime, agent principal, campaign, repository integration or provider connection.	M / R1	Suspension blocks new operations and invalidates relevant short-lived credentials/leases.

16.6 Coordination state models and invariants

16.6.1 Execution campaign states

State	Meaning	Allowed next states (illustrative)
Draft	Campaign is being configured; no runtime may write.	Prepared, Cancelled
Prepared	Baseline, codebase targets and policies pass readiness checks.	Mobilizing, Paused, Cancelled
Mobilizing	Agents/runtimes are being invited, authorized and assigned.	Executing, Prepared, Paused, Cancelled
Executing	One or more authorized work packages are active.	Blocked, Integrating, Paused, Validating, Failed, Cancelled
Blocked	Progress depends on unresolved scope, dependency, policy or provider issue.	Executing, Paused, Failed, Cancelled
Integrating	Change sets are undergoing checks/review/merge queue processing.	Executing, Validating, Blocked, Failed, Cancelled
Validating	Campaign-level acceptance criteria are being evaluated.	Completed, Executing, Blocked, Failed
Paused	New dispatch/write actions are suspended; active work is frozen or safely drained per policy.	Mobilizing, Executing, Integrating, Cancelled
Completed	All required campaign acceptance criteria are satisfied and recorded.	Terminal; a new campaign/baseline is required for further planned work
Failed / Cancelled	Campaign ended without successful completion.	Terminal; recovery uses a new or explicitly resumed campaign under policy

16.6.2 Work-package and lease rules

Object	Required state/constraint	Control rule
WorkPackage	State progression is explicit; terminal outcomes are Integrated, Cancelled or approved Won't Do.	A task is not Integrated until the integration result is recorded; Submitted is not equivalent to merged.
TaskLease	Lease includes owner runtime, work package/resource scope, issued time, expiry, token ID and fencing/version number.	Every write-capable action validates the current lease/fencing number; old lease holders are rejected after expiry/reassignment.
AgentRuntimeInstance	Registered, Authenticating, Online, Busy, Stale, Offline, Suspended or Revoked.	A runtime cannot claim tasks while suspended/revoked or outside permitted campaign scopes.
WorkspaceReference	Prepared, Active, Submitted, Stale, Closed or Invalid.	Workspace is bound to one base commit and assigned scope; stale branch requires rebase/revalidation policy before integration.
IntegrationQueueItem	Pending, ChecksPending, ReviewPending, Ready, Merging, Merged, Rejected, Conflict or Cancelled.	Only Ready items may enter Merging; queue serializes merges for the same target branch.

16.6.3 Business invariants

- MCO-BR-01: Every ExecutionCampaign references one immutable PlanBaseline; all execution records identify the baseline used.
- MCO-BR-02: A campaign target identifies an external repository and integration branch; source code remains authoritative in that external provider.
- MCO-BR-03: An agent principal may have multiple runtime instances, but each runtime session is independently authenticated, scoped and revocable.
- MCO-BR-04: A ParticipationOffer is not a grant. No task or repository access is given until campaign authorization and assignment succeed.
- MCO-BR-05: One exclusive work package/resource has at most one active write lease. Partitioned concurrent writers must have validated non-overlapping scopes.
- MCO-BR-06: A lease is time-bounded and fenced. An expired or revoked runtime cannot commit a valid coordination state transition using an old lease.
- MCO-BR-07: Concurrent writers use separate workspaces/branches or an explicitly approved partitioned/shared-session protocol; no assumption of a shared filesystem is permitted.
- MCO-BR-08: Protected target branches are updated only through the repository policy and IntegrationQueue when the campaign requires controlled integration.
- MCO-BR-09: Submitted, reviewed, tested, merged and verified are separate states; agent self-report does not establish integration or goal completion.
- MCO-BR-10: A change to the locked plan creates a change record and, when approved, a new baseline or controlled migration; historical work retains its original baseline reference.
- MCO-BR-11: Multi-location operation shall not rely on trusted network location; identity, session security and authorization are required for every runtime.
- MCO-BR-12: Coordination events and provider notifications are processed idempotently; retries must not duplicate assignments, merges or accepted outputs.
- MCO-BR-13: A code-host outage or unavailable runtime may pause or block progress but shall not cause the platform to report success.
- MCO-BR-14: Platform telemetry shall not capture source-code content, patch bodies, secrets or raw code-bearing transcripts.

16.7 Non-functional requirements for coordination

The following are initial engineering targets for the R1 coordination pilot. They shall be load-tested and confirmed against an agreed workload profile before contractual SLA commitments.

ID	Quality requirement	Target / condition	Verification
NFR-041	Coordination API availability.	Target ≥ 99.5% monthly for campaign/assignment/lease APIs, excluding approved maintenance.	Synthetic checks and monthly SLI.
NFR-042	Work-package dispatch latency.	95% of eligible ready packages dispatched or made claimable within 5 seconds under the agreed baseline load.	Dispatch latency histogram under load.
NFR-043	Coordination status freshness.	95% of accepted runtime status/heartbeat events reflected in the campaign projection within 10 seconds, excluding provider outages.	Event-to-projection latency test.
NFR-044	Lease safety.	A revoked/expired lease shall be rejected for new write-authorizing operations within 5 seconds of authoritative revocation/expiry processing.	Race and stale-token tests.
NFR-045	Heartbeat detection.	Default heartbeat interval and timeout shall be configurable; initial profile shall detect a lost runtime within 2 minutes after the last accepted heartbeat.	Network fault injection.
NFR-046	Duplicate prevention.	At-least-once delivery/retry shall not create duplicate logical assignments, accepted submissions or merge transitions.	Replay and duplicate webhook test.

ID	Quality requirement	Target / condition	Verification
NFR-047	Initial coordination capacity.	Pilot target: at least 100 concurrent active runtime sessions and 500 registered agent/runtime identities platform-wide; final limits are capacity-profiled.	Load/soak test and quota telemetry.
NFR-048	Horizontal scale.	Stateless coordination APIs and dispatch/telemetry workers shall scale horizontally; lease ownership remains consistent across instances.	Scale-out and contention test.
NFR-049	Network resilience.	Temporary runtime disconnect shall not corrupt campaign state; reconnect shall reconcile last accepted sequence/checkpoint and lease validity.	Disconnect/reconnect chaos test.
NFR-050	Provider resilience.	Code-host/CI provider timeouts shall use bounded retries, backoff, circuit breaking and visible degraded state; no silent merge success.	Provider outage simulation.
NFR-051	Integration serialization.	Merges to the same protected target branch shall be serialized or provider-queue-controlled and revalidated against the latest accepted target.	Concurrent merge race test.
NFR-052	Auditability.	Every campaign state, assignment, lease and integration transition shall be attributable and correlated; normal application roles cannot rewrite the audit trail.	Audit integrity and trace test.
NFR-053	Remote device security.	Runtime communication shall use modern TLS and short-lived, scoped credentials; device/location alone shall not grant trust.	Security configuration and penetration test.
NFR-054	Code boundary under coordination.	Branch, commit, PR/MR and check metadata may be retained; source files, patch/diff bodies and code-bearing raw output shall not be stored or indexed.	Repository connector and agent transcript inspection.
NFR-055	Campaign scale.	The service shall support a configurable campaign with at least 100 work packages and 20 participating runtime instances in the pilot profile.	Campaign simulation and soak test.
NFR-056	Recovery and recovery-point integrity.	Recovery shall restore campaign, lease and queue state consistently with durable events; ambiguous in-flight operations shall be marked for reconciliation, not assumed successful.	Crash/restart and database restore exercise.

16.8 API and event contract requirements

The Coordination Plane shall expose protocol-neutral REST/OpenAPI operations. Each project shall also have a logical project-scoped MCP service, mandatory for standardized real-time agent-to-agent collaboration. MCP tools shall wrap the canonical coordination domain services; they shall not implement independent business rules or become the sole interface for all platform capabilities. The service shall expose authorized tools and resources, and shall support the applicable MCP subscription mechanism for change notification. The exact operation and resource schemas are defined in Section 17 and Appendix G.

API family	Illustrative operations	Required controls
Plan and campaign	POST /plans/{id}/lock; POST /campaigns; GET /campaigns/{id}; POST /campaigns/{id}/pause|resume|cancel	Owner/approver permissions; idempotency; immutable baseline; audit.
Codebase targets	POST /projects/{id}/codebase-targets; GET /codebase-targets/{id}; POST /campaigns/{id}/targets	Provider-scoped credential reference; repository allowlist; protected branch config.
Agent runtime	POST /agent-runtimes/register; POST /agent-runtimes/{id}/heartbeat; POST /agent-runtimes/{id}/offline	Runtime identity; short-lived token; nonce/sequence; revocation.

API family	Illustrative operations	Required controls
Participation/scheduling	POST /campaigns/{id}/participation-offers; POST /offers/{id}/accept; GET /campaigns/{id}/eligible-work	Offer ≠ authorization; capability matching; policy decision.
Work packages and leases	GET /work-packages/{id}; POST /work-packages/{id}/claim; POST /leases/{id}/renew; POST /leases/{id}/release	Lease token/fencing number; scope validation; optimistic concurrency.
Workspace/change set	POST /work-packages/{id}/workspaces; POST /work-packages/{id}/submissions; GET /change-sets/{id}	External references only; base commit; idempotency; code payload rejection.
Integration queue	POST /integration-queue; GET /integration-queue/{id}; POST /integration-queue/{id}/approve|reject; GET /campaigns/{id}/integration-status	Approver permissions; policy checks; serialized target-branch merge.
Campaign telemetry	GET /campaigns/{id}/status; GET /campaigns/{id}/events; GET /campaigns/{id}/dependencies	Authorization before filtering; pagination; no secret/code payloads.

The platform shall emit versioned events, including CampaignCreated, PlanBaselineLocked, RuntimeRegistered, ParticipationOffered, ParticipationAccepted, WorkPackageAssigned, LeaseIssued, LeaseRenewed, LeaseExpired, RuntimeHeartbeatMissed, WorkspaceRegistered, WorkPackageBlocked, ChangeSetSubmitted, IntegrationChecksCompleted, IntegrationQueued, ChangeIntegrated, ChangeRejected, CampaignPaused, CampaignResumed and CampaignCompleted. Event consumers shall be idempotent and preserve event/correlation/causation identifiers.

16.9 Security, privacy and source-code boundary

- Runtime authentication shall be based on machine identity and short-lived session credentials. Credentials shall be revocable independently of the durable Agent Principal.
- A task lease is an authorization-bound coordination token, not a substitute for repository-provider authorization. External code-host tokens shall be least-privilege and limited to the required repository/branch/actions.
- A local laptop or remote device shall not be trusted merely because it is in a known network or geographic location. Device posture/attestation may be added by policy but shall be an explicit capability.
- The platform shall not transmit an agent a repository credential broader than the campaign target and task require. Where possible, repository operations shall be mediated by a scoped integration service or short-lived provider credential.
- Runtime logs and telemetry shall capture operational facts (session, task, timing, result, error category) without source-code bodies, secrets, access tokens or full prompts containing restricted/code data.
- Agent participation is opt-in/authorized per campaign. Agent registration alone does not imply permission to inspect a project, join a campaign, access a repository or execute a task.
- A human approver shall be configurable for plan lock, re-baselining, protected-branch integration and production-impacting actions. Autonomous approval shall be explicitly enabled, narrowly scoped and audited if ever supported.

16.10 Verification and acceptance scenarios

16.10.1 Primary distributed execution scenario

1. Create a software project with an approved and locked plan containing at least four work packages, explicit dependencies and acceptance criteria.
2. Register one external repository and capture its canonical repository ID, protected target branch and initial commit SHA.
3. Create an ExecutionCampaign tied to the locked baseline and repository target.
4. Register at least two AgentRuntimeInstances for authorized agents on different devices or network locations. Ensure they have distinct session identities and only scoped repository/task permissions.
5. Submit participation offers; authorize both agents; assign independent or explicitly partitioned work packages with non-overlapping write scopes.
6. Issue task leases and isolated branch/workspace references based on the same recorded repository baseline.
7. Run both agents concurrently. Record heartbeats, progress, blockers, context snapshot IDs, branch references, commit IDs and sanitized summaries.
8. Submit external PR/MR references. Verify checks and approvals. Place candidates in the integration queue and merge sequentially into the protected target branch.
9. Record resulting merged commit SHAs and verification evidence. Reconcile dependent packages and campaign-level acceptance criteria.

10. Confirm the platform shows one goal, one plan baseline, one repository identity, multiple agent sessions, isolated workspaces, ordered integration history and evidence-backed campaign completion.
11. Inspect the database, object storage, search index, logs and event payloads to confirm source code and diffs were not retained.

16.10.2 Failure and concurrency scenarios

Scenario	Expected behavior
Two agents claim the same exclusive work package	Only one receives the active write lease; the other receives a conflict/queue response.
Two agents work on disjoint packages in the same repository	Both may execute concurrently in isolated branches/workspaces based on the recorded base commit.
Two submissions overlap on a reserved path/resource	Coordinator flags overlap and blocks, serializes or requests explicit resolution according to policy.
Runtime loses network connectivity	Heartbeat becomes stale; the lease expires or is revoked; workspace/checkpoint references remain; reassignment requires reconciliation.
Plan changes after campaign starts	Original baseline remains immutable; affected work is assessed; approval creates a new baseline or controlled migration.
CI checks fail	Integration remains blocked/rejected; work package is not marked Integrated.
External merge succeeds but webhook is delayed/replayed	Provider reconciliation and idempotency identify the one merge result; no duplicate transition is created.
Code-host provider is unavailable	Campaign enters degraded/blocked state; no false success or fabricated merge result is shown.
Agent principal is revoked during work	New operations are denied; active session and lease are revoked; in-flight external workspace is flagged for owner review.

16.11 Release strategy and implementation sequence

Multi-agent shared-codebase coordination is committed to R1 because it is the primary commercial product wedge and depends on the foundation MVP knowledge, identity, provenance, agent context and integration capabilities. R1 shall be delivered in controlled increments:

R1 increment	Scope	Exit condition
R1-A: Plan lock and campaign baseline	PlanBaseline, approval/lock action, repository target registry, ExecutionCampaign and immutable references.	A campaign can be created reproducibly from approved planning state and a pinned codebase commit.
R1-B: Remote agent sessions and work packages	AgentRuntimeInstance, participation offers, capabilities, work-package mapping, dependency DAG, assignment and context snapshots.	Two remote runtimes are authorized and assigned coherent packages tied to one goal/baseline.
R1-C: Leases and workspace isolation	Heartbeat, task/resource leases, fencing, external branch/workspace adapter and recovery/reassignment.	Parallel work can proceed without shared-write races; disconnect/revocation is safe.
R1-D: Integration queue and evidence	PR/MR references, CI checks, serialized integration, approvals, merged commit refs and campaign dashboard.	Changes are integrated into the shared external codebase through the governed queue and campaign status is evidence-backed.

R1 shall not be declared complete solely because agent assignment and progress displays exist. It must demonstrate concurrent distributed execution, lease safety, isolated workspaces, controlled integration, failure recovery and preservation of the source-code boundary.

16.12 Open design decisions for the Coordination Domain Specification

Decision	Recommended starting point	Must be finalized before
Initial code-host support	Select one provider for the first R2 pilot; design a provider-neutral adapter contract.	R2 design freeze
Remote runtime transport	Outbound authenticated worker connection with heartbeat/sequence; support polling fallback where needed.	Runtime SDK/agent adapter
Workspace strategy	One isolated branch/worktree per writing work package; provider-managed workspace adapters optional.	Workspace adapter implementation

Decision	Recommended starting point	Must be finalized before
Lease defaults	Configurable heartbeat/lease policy; start with a 30-second heartbeat and 2-minute stale detection target, with fencing tokens.	Pilot load/failure testing
Concurrency policy	Exclusive writer by default; partitioned parallel writers only with validated non-overlap; review-only agents may collaborate freely.	Scheduler and lease implementation
Merge policy	Protected target branch, serialized integration queue, required CI and configurable human approval.	Code-host integration implementation
Task decomposition	Owner-approved work breakdown initially; later allow AI proposals that require validation/approval before dispatch.	R2-A/B
Runtime trust	Short-lived credentials and project/repository/task scope; add device posture/attestation based on threat model.	Remote runtime pilot
Plan re-baselining	New immutable baseline; explicit impact assessment and owner decision for each in-flight package.	Change-control workflow

16.13 Updated glossary additions

Term	Definition
Coordination Plane	Platform capability that schedules and coordinates distributed agent work, leases, workspaces and integration while source code remains external.
Execution Campaign	A coordinated implementation effort tied to a locked plan baseline and defined codebase targets.
PlanBaseline	Immutable snapshot of approved plan inputs and exact knowledge revisions used by a campaign.
Agent Runtime Instance	A particular execution session/device/worker instance of a durable Agent Principal.
Task Lease	Time-bounded, scoped ownership of work or a logical resource; enforced with expiry and fencing.
Workspace Isolation	Separation of each concurrent writer working copy/branch/worktree to avoid simultaneous modification of one shared working directory.
Integration Queue	Controlled and serialized path for validating and merging external change sets into a protected target branch.
Fencing Token	Monotonically changing lease/version value used to reject operations from stale lease holders.

16.14 Normative conclusion

The platform shall support coordinated, distributed, multi-agent implementation as a first-class capability. Multiple authorized agents may work from different locations and devices toward one shared goal and one authoritative external codebase after planning has been locked. Safe parallelism is achieved through plan-bound work packages, capability-aware assignment, authenticated runtime sessions, leases/resource reservations, isolated workspaces/branches, dependency management and controlled integration—not by allowing uncoordinated concurrent writes to the same working directory. The Coordination Plane shall preserve the complete execution and integration history while maintaining the strict source-code exclusion boundary. Each project shall provide a project-scoped MCP coordination service through which authorized agents can discover participants, exchange durable messages, negotiate work, subscribe to coordination changes and recover missed events. The service shall mediate communication; it shall not depend on direct agent-to-agent connectivity.

[R11] Model Context Protocol maintainers (2026). Model Context Protocol Specification, 2026-07-28. https://github.com/modelcontextprotocol/modelcontextprotocol/tree/main/docs/specification/2026-07-28 Protocol baseline for the project MCP service; verify SDK and compatibility at implementation kickoff.
[R12] Model Context Protocol maintainers (2026). Server Resources, 2026-07-28. https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/server/resources.mdx Resource subscriptions and change notifications; DevWeave adds durable event storage and replay.
[R13] Model Context Protocol maintainers (2026). The 2026-07-28 Specification. https://blog.modelcontextprotocol.io/posts/2026-07-28/ Published specification release overview.

Appendix F. Coordination Data Concepts and State Models

The following is an illustrative logical object map. Physical tables and API payloads shall be specified in the Coordination Domain Specification.

Object	Key relationships
ExecutionCampaign	belongs to Project; created from PlanBaseline; targets one or more CodebaseTargets; contains WorkPackages, AgentRuntime participation and IntegrationQueueItems.
PlanBaseline	references immutable KnowledgeItem revisions, relationship/dependency snapshot, lifecycle profile version, objective and acceptance policy.
AgentRuntimeInstance	belongs to Agent Principal; participates in campaigns; owns active TaskLeases; reports CoordinationEvents.
WorkPackage	belongs to Campaign and PlanBaseline; depends on other WorkPackages; is assigned to runtime; references Knowledge context, CodebaseTarget and WorkspaceReference.
TaskLease	belongs to WorkPackage or ResourceReservation; issued to AgentRuntimeInstance; expires/revokes; carries fencing/version token.
WorkspaceReference	belongs to WorkPackage and runtime; targets external repository and base commit; may produce ChangeSetReference.
ChangeSetReference	belongs to WorkPackage/campaign; references external commit/branch/PR/MR; enters IntegrationQueueItem; is validated by Evidence/CI records.
IntegrationQueueItem	targets protected branch; records checks, reviews, queue order, merge state and resulting commit SHA.
CoordinationEvent	references campaign/project and relevant actor/runtime/work package/lease/change-set; append-only and correlation-aware.

Illustrative execution relationship: PlanBaseline → ExecutionCampaign → WorkPackages → AgentRuntimeInstances/TaskLeases → WorkspaceReferences → ChangeSetReferences → IntegrationQueueItems → Merged Commit References → Verification Evidence → Campaign Acceptance.

17. Project-Level MCP Service and Real-Time Agent-to-Agent Coordination

17.1 Purpose and requirement status

This section adds a mandatory coordination capability to DevWeave. In addition to preserving project knowledge and coordinating distributed implementation work packages, the platform shall provide a logically project-scoped Model Context Protocol (MCP) service that enables authenticated agent runtimes participating in the same project or execution campaign to communicate and coordinate with one another in real time.

The service is a platform-mediated coordination mechanism. Agent A submits a message or coordination action to the project MCP service; the service authorizes and durably records the action; subscribed Agent B clients receive a change notification and retrieve the corresponding event/message. Agents do not need direct network reachability, shared filesystems or direct trust relationships.

The MCP interface is mandatory for standardized real-time agent-to-agent collaboration. REST/OpenAPI remains available for user applications, administrative operations, integrations, reporting and non-MCP clients. MCP tools shall call canonical domain services; they shall not create a second source of truth.

Priority: M / R1 for delivery of the first commercial coordinated-agent collaboration capability. A logical project-scoped MCP service identity, endpoint reference, resource namespace and policy record shall exist for every project from the project model; real-time coordination capabilities are delivered in R1. The implementation may host many logical project services on a shared, horizontally scaled service fleet; a dedicated process or server deployment for every project is not required.

17.2 Definitions

Term	Definition
Project MCP Service	A logically project-scoped MCP endpoint, capability catalogue, resource namespace and authorization policy through which agents access project coordination functions.
MCP client / agent runtime	An authenticated software agent instance that calls the project MCP service and may maintain one or more notification subscriptions.
Coordination channel	A scoped communication stream for a project, execution campaign, work package, team or direct agent-to-agent exchange.
Coordination message	A durable, typed, attributable message or structured proposal stored by the platform and delivered to authorized recipients.
Coordination event	An immutable event describing a change in presence, message, proposal, handoff, blocker, assignment, lease or campaign state.
Presence	A time-bounded, approximate indication of the last authenticated activity and reported availability of an agent runtime.
Cursor	An opaque or monotonic position within a channel/event stream used to resume reading after interruption.
Notification	A best-effort MCP signal indicating that a subscribed resource changed; it is not itself the authoritative message or event record.
Handoff	A proposed transfer of a task or responsibility between agents which requires explicit acceptance and authoritative lease/assignment update.

17.3 Service boundary and deployment model

- The platform shall provision or make available a logical MCP service identity and resource namespace for each project. The service may be disabled only by an explicit tenant/project policy; disabling it shall not delete historical coordination records.
- A shared MCP service fleet may serve multiple tenants and projects. Every request shall identify the project context explicitly or derive it from a verified opaque endpoint plus the authenticated grant. The service shall not rely on connection affinity to infer the project, agent, campaign or conversation.
- The service shall integrate with the canonical Identity, Tenancy, Project, Agent, Coordination, Governance, Provenance and Audit modules. It shall not connect directly to PostgreSQL from third-party agents and shall not expose arbitrary SQL, filesystem access or unrestricted repository operations.
- Project configuration shall control the enabled coordination capabilities, channel types, allowed agent classes, message schemas, retention rules, subscription policies, rate limits and integration capabilities.

- A project may have multiple campaigns and many agent runtime instances. A single agent principal may operate multiple runtime instances, each with its own authenticated runtime identity, presence and permissions.
- The project MCP endpoint shall be stable and discoverable by authorized project participants. Resource URIs shall be opaque or non-sensitive and shall not expose secret tenant identifiers.

17.4 Real-time coordination architecture

The service shall use a durable store-and-forward architecture. The durable coordination message/event store is authoritative. An internal event dispatcher or broker fans out change notifications to active MCP subscriptions. The MCP service may be stateless across requests and horizontally scaled; coordination state shall be stored in durable services and referenced using explicit IDs.

Component	Responsibility	Required behaviour
MCP endpoint / protocol adapter	Expose project-scoped tools, resources and subscription support.	Validate protocol version/capabilities; authenticate every request; map protocol calls to domain services.
Coordination domain service	Authorize participants, manage channels, messages, presence, proposals, handoffs and acknowledgements.	Apply project/channel policy; persist before success acknowledgement; enforce idempotency and sequence allocation.
Durable message/event store	Preserve ordered channel messages and coordination events.	Provide replay by cursor; retain actor, timestamps, correlation and delivery state; survive process restart.
Event dispatcher / broker	Fan out resource-change signals to subscribed MCP streams.	Support multi-instance dispatch; use outbox or equivalent reliable publication; tolerate retries and duplicate signals.
Agent runtime subscription manager	Maintain subscriptions and react to updates on participating devices/runners.	Reconnect with backoff; read missed events by cursor; acknowledge progress; do not assume a notification was delivered.
Policy and audit services	Apply tenant/project/resource policies and record sensitive coordination actions.	Prevent cross-scope disclosure; record send, read, handoff, mute, revoke and moderation actions.

The internal dispatcher may initially use a transactional outbox and an in-process or broker-backed fan-out implementation. A durable broker such as a stream/log service may be introduced when measured scale requires it. A transient pub/sub mechanism alone shall not be the system of record.

17.5 Agent communication model

The platform shall support service-mediated communication between authorized agent principals and runtime instances. Communication shall be possible across different devices, network locations, cloud regions and execution environments, subject to the platform security policy and supported transport availability.

Channel type	Purpose	Membership rule
Project	Project-wide coordination, announcements and shared context.	Explicit project role/permission; may be restricted to selected agent classes.
Campaign	Coordination among agents contributing to one locked execution campaign.	Authorized campaign participants and assigned reviewers.
Work-package thread	Clarifications, handoffs, blockers and coordination for one work package.	Assigned agent runtimes, task owner and authorized reviewer.
Team / specialist group	Communication among agents with a shared role or subsystem responsibility.	Explicit group membership and project policy.
Direct agent-to-agent	One-to-one messages, questions or requests for help.	Both sender and recipient must be authorized project participants; policy may restrict direct messaging.

A message may be typed as announcement, intent, question, answer, proposal, acceptance, rejection, handoff request, blocker, conflict notice, status update or decision reference. Message types and payload schemas shall be registered and versioned. A message does not itself grant task, repository, approval or execution authority.

Messages shall carry sender principal/runtime identifiers, channel ID, server-generated sequence, message ID, creation time, optional reply/parent ID, correlation ID, idempotency key, content schema version and classification. The service shall distinguish persisted, published, notified, read and acknowledged states. A sender shall not be told that a recipient read or accepted a message unless that event is recorded.

17.6 MCP capabilities and interaction contract

The project MCP service shall expose MCP tools for coordination commands and MCP resources for current coordination state and event/message streams. Project-specific prompts may be provided to explain coordination conventions, message schemas and safe collaboration rules. Tools and resources shall be filtered according to the authenticated principal and project permissions.

MCP capability	Initial operation/resource	Purpose
Tool	devweave_coordination_discover_agents	List only authorized participating agents and permitted capability/availability metadata.
Tool	devweave_coordination_get_presence	Read current and last-seen presence for an authorized participant.
Tool	devweave_coordination_join_channel	Request or confirm channel membership where policy permits.
Tool	devweave_coordination_send_message	Persist a typed message to a channel or authorized recipient.
Tool	devweave_coordination_read_messages	Read messages/events after a supplied cursor with bounded pagination.
Tool	devweave_coordination_acknowledge	Record a consumer processed-through cursor or message acknowledgement.
Tool	devweave_coordination_propose_work	Propose a work split, dependency, pairing session or task handoff without silently changing authoritative assignment.
Tool	devweave_coordination_request_handoff	Create a handoff request to an eligible agent or group.
Tool	devweave_coordination_accept_handoff	Accept a handoff; assignment/lease changes are then performed by the authoritative Coordination Plane.
Tool	devweave_coordination_raise_blocker	Create a durable blocker or conflict event and notify authorized participants.
Tool	devweave_coordination_publish_status	Publish a bounded status update for a runtime, work package or campaign.
Resource	devweave://projects/{projectId}/coordination/overview	Current project coordination summary and authorized participant overview.
Resource	devweave://projects/{projectId}/campaigns/{campaignId}/participants	Authorized campaign roster and presence snapshot.
Resource	devweave://projects/{projectId}/coordination/channels/{channelId}/events	Ordered channel messages and coordination events, retrievable from a cursor.
Resource	devweave://projects/{projectId}/work-packages/{workPackageId}/coordination	Task-specific coordination state, blockers, handoffs and related message references.
Subscription	subscriptions/listen with resourceSubscriptions	Opt in to resource-change notifications for authorized coordination resources.
Prompt (optional)	devweave://projects/{projectId}/coordination/protocol	Describe project collaboration conventions and safe agent interaction rules.

The operation names above are DevWeave application-level tool names, not new MCP protocol methods. They shall be exposed through the standard MCP tool mechanism. Project resource URIs shall be read through standard MCP resource operations. Subscribed clients shall use the protocol-defined subscriptions/listen mechanism supported by the negotiated MCP protocol version.

17.7 Functional requirements

The following requirements are mandatory for the project-level MCP coordination capability. M / R1 denotes mandatory for the first commercial coordinated-agent release. Each requirement is independently verifiable.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
MCP-001	The system shall provision a logical project-scoped MCP service identity, stable endpoint reference, capability catalogue and resource namespace for every project. The service may be disabled only by an explicit authorized project/tenant policy; disabling it shall not delete its configuration history or coordination records.	M / R1	Create two projects and verify each has a distinct logical MCP service identity/resource namespace and authorization boundary; verify explicit disablement is audited and historical data remains retained.
MCP-002	Each MCP service request shall be authenticated and authorized against the verified tenant, project, principal, runtime and requested resource/action.	M / R1	An unauthorized agent cannot list, read or mutate another project's resources.
MCP-003	The project MCP service shall expose coordination tools and resources through the negotiated supported MCP protocol version.	M / R1	Conformance tests discover and invoke the documented tool/resource catalogue.
MCP-004	The system shall support discovery of authorized project/campaign participants, including only policy-approved identity, role, capability and availability metadata.	M / R1	An agent sees eligible participants but no hidden project members or confidential profile attributes.
MCP-005	The system shall maintain runtime presence using authenticated heartbeats or equivalent activity signals and shall classify presence as approximate and time-bound.	M / R1	Disconnect a runtime and verify that its presence becomes stale/offline after configured expiry.
MCP-006	The system shall support project, campaign, work-package, team and policy-controlled direct agent-to-agent channels.	M / R1	Create each enabled channel type and verify the applicable membership policy.
MCP-007	The system shall authorize channel creation, join, leave, mute, archive and membership changes according to project policy.	M / R1	Unauthorized membership operations are rejected and audited.
MCP-008	An authorized agent shall be able to send a typed coordination message to an authorized channel or recipient through an MCP tool.	M / R1	Agent A sends to Agent B on a separate runtime; the service returns a durable message ID and channel sequence.
MCP-009	The system shall durably persist a coordination message and its actor, runtime, channel, timestamp, schema version, correlation and idempotency metadata before acknowledging successful submission.	M / R1	After an acknowledged send, restart service processes and retrieve the same message.
MCP-010	The system shall assign a monotonic sequence within each channel and preserve deterministic ordering for messages committed to that channel.	M / R1	Concurrent sends produce a unique ordered channel sequence; no global order across channels is required.
MCP-011	The system shall support cursor-based bounded reading of messages and events for a channel or coordination resource.	M / R1	Read from a supplied cursor and receive the next ordered page and continuation cursor.
MCP-012	The service shall support MCP resource subscriptions for authorized coordination resources and shall emit resource-change notifications when their visible state changes.	M / R1	A subscribed agent receives a resource update signal after a message is committed.
MCP-013	The system shall treat MCP notifications as best-effort signals and shall provide durable read/replay so missed notifications do not cause message or event loss.	M / R1	Disconnect a subscriber, create events, reconnect and retrieve all missing events from its last cursor.
MCP-014	The system shall support duplicate-safe message submission and event consumption through idempotency keys, message IDs and cursors.	M / R1	Replay the same send request and delivery event; no duplicate logical message or state transition is created.
MCP-015	The service shall support explicit acknowledgements or processed-through cursors for subscribed agents where delivery tracking is required.	S / R2	The service records the acknowledged cursor separately from notification dispatch.
MCP-016	The system shall support message replies and threaded coordination using explicit parent/message identifiers.	S / R2	A reply links to an existing authorized message and remains readable in thread context.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
MCP-017	The system shall support work proposals, requests for help, conflict notices and handoff requests as typed coordination records.	M / R1	Create and inspect proposals/handoffs with explicit states and actor history.
MCP-018	A handoff shall not alter authoritative task assignment or lease ownership until an eligible recipient explicitly accepts and the Coordination Plane commits the transition.	M / R1	A proposed but unaccepted handoff leaves the original owner/lease unchanged.
MCP-019	The system shall provide coordination status resources for campaigns and work packages, including participants, blockers, messages and handoff references visible to the requesting agent.	M / R1	Resource read returns a current authorized projection with source event/version metadata.
MCP-020	The system shall not require direct network connectivity between agent runtimes for message exchange or coordination.	M / R1	Two agents behind unrelated NAT/firewall networks communicate only through the project service.
MCP-021	The system shall support disconnected/offline runtimes by retaining authorized messages/events and allowing catch-up after reconnect.	M / R1	Messages sent while a runtime is offline are available on its next authorized cursor read.
MCP-022	The service shall support multiple simultaneous MCP clients and subscriptions for one project, campaign and channel.	M / R1	At least two independent agent clients subscribe to one campaign channel and receive updates.
MCP-023	The system shall prevent unauthorized cross-tenant, cross-project, cross-campaign and cross-channel message/resource disclosure, including through notification metadata.	M / R1	Negative access tests cover tool results, resources, notifications, logs, search and replay.
MCP-024	The system shall rate-limit and apply payload-size limits to message send, read, subscription and presence operations.	M / R1	Exceeding configured limits returns documented errors without degrading unrelated tenants.
MCP-025	Coordination messages shall be treated as untrusted input and shall not automatically execute tools, change plans, approve work, change leases or alter repository state.	M / R1	A message containing an instruction does not execute it or grant authority.
MCP-026	The platform shall record auditable events for message send, read/acknowledgement where enabled, membership change, handoff, moderation, mute, revoke and policy-denied actions.	M / R1	Audit trail identifies the authenticated actor/runtime, target, timestamp and outcome.
MCP-027	The system shall allow authorized project administrators to configure coordination channels, allowed message types, retention, visibility, participant eligibility and service enablement.	M / R1	Policy changes affect subsequent operations and are versioned/audited.
MCP-028	The MCP service shall use explicit project, campaign, channel, message, subscription and cursor identifiers to carry state across calls; it shall not infer durable collaboration state from a connection or process identity.	M / R1	A client can call tools across different server instances using explicit handles.
MCP-029	The system shall support reconnect and subscription renewal with bounded backoff and cursor-based recovery.	M / R1	Interrupt the stream and verify re-subscription and event catch-up without a permanent gap.
MCP-030	The system shall expose a documented fallback read/poll operation for MCP clients that cannot maintain long-lived subscriptions.	M / R1	A polling client retrieves updates with the same cursor semantics and authorization.
MCP-031	The service shall maintain message/event correlation across coordination operations, work packages, execution campaigns and agent activities.	M / R1	A message related to a task can be traced to campaign, work package and agent execution context.
MCP-032	The system shall enforce source-code exclusion and secret filtering on MCP message bodies, resource payloads, logs and event metadata.	M / R1	Code/diff/secret test payloads are rejected, quarantined or redacted before durable persistence.
MCP-033	The MCP service shall advertise and negotiate supported protocol versions and capabilities and shall maintain a documented compatibility/deprecation policy.	M / R1	Protocol conformance suite validates supported versions and rejects incompatible requests safely.

ID	Requirement (shall statement)	Priority / release	Verification / acceptance
MCP-034	The service shall support graceful revocation of an agent principal/runtime and terminate or deny its subsequent tool calls and resource subscriptions.	M / R1	Revoke a runtime and verify tool, read, subscription and message-send access is removed within the configured revocation target.
MCP-035	The service shall distinguish message persistence, notification dispatch, recipient read, recipient acknowledgement and recipient acceptance as separate states.	M / R1	User interface/API tests show no false delivery/read/acceptance claim.
MCP-036	The project MCP service shall provide documented operational health and subscription diagnostics without exposing message content or credentials to unauthorized operators.	S / R2	Authorized admin inspects health, lag and failure counts; sensitive payloads remain unavailable.

17.8 Message and event data requirements

The logical schema shall be extensible and versioned. The following conceptual entities may be separate relational records or compatible aggregates; physical decomposition is determined by the Coordination Domain Specification.

Entity	Minimum attributes	Rules
ProjectMcpServiceConfig	projectId, serviceId, endpointRef, protocolProfile, enabled, capabilitySet, policyVersion, retentionPolicyId	One active logical configuration per project; versions are auditable; endpoint secrets are not stored in plaintext.
CoordinationChannel	channelId, tenantId, projectId, channelType, campaignId?, workPackageId?, visibility, status, sequenceHighWatermark	Channel membership and visibility are explicit; scope cannot cross tenant/project boundary.
CoordinationMessage	messageId, channelId, channelSequence, senderPrincipalId, runtimeInstanceId, messageType, payloadSchemaVersion, payloadRef/body, parentMessageId?, correlationId, idempotencyKey, createdAt	Append-only after commit; size and content policies apply; unique (channelId, channelSequence) and idempotency constraints.
CoordinationEvent	eventId, projectId, campaignId?, channelId?, eventType, actorId, targetRef?, sequence/cursor, recordedAt, correlationId, payload	Append-only, ordered within applicable stream, replayable and deduplicable.
AgentPresence	principalId, runtimeInstanceId, projectId, state, capabilities, availability, lastHeartbeatAt, expiresAt	Ephemeral projection derived from authenticated activity; never treated as proof of agent correctness or identity by itself.
CoordinationSubscription	subscriptionRef, principalId, runtimeInstanceId, resourceUris, openedAt, lastSeenAt, status	Transport/request scoped where required; reconnect creates a new subscription and reuses durable cursor handles.
CoordinationCursor	principalId, channelId/resourceId, lastReadSequence, lastAcknowledgedSequence, updatedAt	Cursor is scoped; advancement cannot skip unread records unless an explicit consumer policy permits it.
HandoffRequest	handoffId, sourceAgentId, targetAgentId/groupId, workPackageId, reason, status, proposedAt, decidedAt	Proposal and acceptance are distinct; assignment/lease change is a separate authoritative transition.

17.9 Payload schema and source-code boundary

- Coordination message payloads shall use registered, versioned schemas. The default message body shall support concise text and structured metadata, with references to existing KnowledgeItems, WorkPackages, Evidence, Activities or external resources.
- Binary content shall be sent as an authorized artifact reference, not embedded as an unrestricted binary payload. Artifact scanning, classification and source-code exclusion policies shall apply.
- Source files, patch bodies, code diffs, executable payloads, credentials, secrets and raw code-bearing prompts/transcripts shall not be preserved in coordination messages or event payloads. The ingestion path shall reject, redact or quarantine detected prohibited content before persistence.
- Coordination messages from other agents are untrusted data. Agents and hosts shall not interpret a message as a system instruction, approval, policy override or authorization grant. Each recipient runtime remains responsible for safe prompt construction and tool gating.
- The sender identity used for authorization shall be obtained from validated credentials or trusted runtime attestation, not from a message body or self-reported MCP clientInfo field.

17.10 Delivery, ordering, presence and recovery semantics

Concern	Normative semantics
Persistence	A successful send result means the message and its event/outbox record are durably committed. It does not mean any recipient has received or read it.
Ordering	The service shall allocate a monotonically increasing sequence per channel. It shall not promise a single total order across independent channels.
Notification	MCP resource-update notifications are best-effort signals. They may be delayed, duplicated or missed across disconnects. They shall not be the only copy of the content.
Read/replay	A client shall retrieve messages/events using the channel resource and a last-seen cursor. Reconnect shall replay from the last successfully processed cursor.
Delivery model	The platform shall provide at-least-once availability through durable replay; clients shall deduplicate by event/message ID and idempotently process updates. Exactly-once network delivery shall not be claimed.
Acknowledgement	Read and acknowledgement are explicit and separate from notification dispatch. A consumer may record a processed-through cursor.
Presence	Presence is approximate. The platform shall report last heartbeat and freshness rather than assert continuous human/agent attention.
Handoff	Handoff follows propose → accept/reject → authoritative assignment/lease transition. Timeout or rejection leaves ownership unchanged unless a recovery policy explicitly reassigns it.
Backpressure	Slow subscribers shall not block message persistence or unrelated tenants. The service shall bound queues, signal lag, and rely on replay rather than unbounded in-memory buffering.
Recovery	Outbox/event dispatch, subscription reconnect, runtime rejoin and service restart shall preserve durable records and allow cursor recovery.

17.11 Protocol conformance and compatibility

The SRS protocol baseline is the published MCP specification dated 2026-07-28 as reviewed on 09 November 2026. Implementation shall re-check the official specification and SDK support at engineering kickoff; this version reference is a baseline, not permission to implement draft protocol behaviour.

- The service shall conform to the negotiated MCP protocol version and advertise the capabilities it actually implements. It shall not expose undocumented custom JSON-RPC methods as if they were standard MCP methods.
- For the 2026-07-28 protocol profile, requests are stateless. Every request shall carry required protocol metadata and explicit application handles. The server shall not infer project, campaign, message or conversation continuity from a TCP connection, HTTP connection, process or previous request.
- The service shall use the standard MCP tools/resources model. For resource-change updates it shall support the protocol-defined subscriptions/listen mechanism and resourceSubscriptions filter where negotiated. A resource update notification signals that the resource changed; the client then reads the current resource or event page.
- The 2026-07-28 MCP specification does not make notifications a durable event log. DevWeave shall therefore maintain durable message/event records and cursor-based replay independently of the notification stream.
- The service shall not rely on deprecated legacy HTTP+SSE or session assumptions for a new implementation. Compatibility with earlier supported agents shall be implemented through explicit, tested adapters or version negotiation, not silent protocol ambiguity.
- Authentication and authorization shall follow the applicable MCP HTTP authorization requirements and DevWeave identity policies. Self-reported clientInfo/serverInfo fields may be used for diagnostics but shall not be treated as authenticated identity.

17.12 Security and governance requirements

- An agent may discover only participants and capability metadata that project policy permits. Presence, direct messages and channel history shall be subject to resource-level authorization.
- A tenant/project administrator shall be able to revoke an agent principal, runtime instance, credential, channel membership or project MCP service capability. Revocation shall prevent new reads/writes and terminate active subscriptions within a measurable operational target.
- Project administrators shall be able to define allowed channel types, recipient classes, retention periods, message schemas, rate limits and moderation procedures.
- The platform shall provide controls to report, mute, block or remove an abusive participant and to preserve relevant audit records without exposing unrelated private messages.

- MCP tool calls that create messages, proposals, handoffs or status changes shall be auditable and idempotent. Any operation that changes task assignment, lease, approval or code integration shall invoke the authoritative domain operation and its own authorization gate.
- Coordination events shall be filtered before fan-out. An event must not leak unauthorized payload fields through notification metadata, resource URI, timing detail, error messages or subscriber diagnostics.

17.13 Non-functional requirements and initial targets

The following are pilot engineering targets for the initial R1 workload profile. They are not contractual service-level commitments until validated by load tests, representative network conditions and operational readiness review.

ID	Quality requirement	Target / condition	Verification
MCP-NFR-001	For a connected subscriber under the pilot workload, the p95 interval from durable message commit to notification dispatch shall be no more than 2 seconds.	Pilot: 50 active runtimes in one project; 500 active runtimes per tenant; 5 messages/second/project sustained.	Instrument commit-to-dispatch latency; report p50/p95/p99.
MCP-NFR-002	The p95 time to acknowledge a valid message after durable commit shall be no more than 1 second under the pilot workload.	Payload <= 16 KB; normal service health; defined region and load.	Load test send operations and verify durable commit boundary.
MCP-NFR-003	An acknowledged message shall survive MCP process restart and event-dispatcher restart.	Durability is measured at the service acknowledgement boundary.	Crash/restart test followed by cursor replay.
MCP-NFR-004	The service shall provide monotonically increasing, unique per-channel sequence values.	Independent channels need not share a global order.	Concurrent producer test verifies no duplicate or regressing sequence.
MCP-NFR-005	After reconnect, a client shall be able to recover all authorized committed messages/events after its last acknowledged cursor.	Subject to configured retention and legal holds.	Disconnect/reconnect and compare source stream with replay output.
MCP-NFR-006	Presence shall be marked stale/offline no later than the configured heartbeat expiry plus 15 seconds of processing allowance.	Initial default heartbeat 15 seconds; expiry target 45 seconds; configurable per runtime class.	Simulate abrupt disconnection and measure state transition.
MCP-NFR-007	A single slow or disconnected subscriber shall not block durable message acceptance or notification delivery for other subscribers.	Bounded per-subscriber buffers and tenant fairness required.	Inject slow consumer and measure unrelated send latency.
MCP-NFR-008	The service shall support horizontal scaling of MCP request handling and event dispatch without requiring sticky routing for ordinary stateless requests.	Explicit application handles and shared durable state.	Run multiple instances behind load balancer; route successive calls to different instances.
MCP-NFR-009	All message/event payloads and notification metadata shall satisfy tenant/project/channel authorization before delivery.	No cross-scope disclosure under concurrency or cache reuse.	Automated multi-tenant negative and cache-isolation tests.
MCP-NFR-010	The service shall enforce an initial configurable 64 KB maximum inline message payload; larger content must be stored as a governed artifact reference.	Payload size measured after serialization; administrators may set a lower limit.	Boundary test at limit and above limit.
MCP-NFR-011	The service shall provide bounded retention, archival and deletion/pseudonymization policy without corrupting retained sequence/cursor history.	Retention is configurable by channel/project and legal hold.	Retention simulation and audit/export verification.
MCP-NFR-012	The service shall expose operational metrics for send rate, commit latency, notification lag, active subscriptions, reconnects, replay volume, queue depth, dropped notifications and authorization denials.	Metrics shall not include message body, secrets or code.	Telemetry review and sensitive-data scan.
MCP-NFR-013	The project coordination service shall meet the platform availability target adopted for the same Release 2 production tier.	Final numeric SLO to be approved under OD-10 and OD-19.	Availability test/report against approved service objective.
MCP-NFR-014	MCP service errors and retries shall be bounded and documented, with correlation IDs and actionable client retry guidance.	No uncontrolled retry storm.	Fault injection and retry/backoff tests.

17.14 Verification and acceptance scenarios

The following scenarios are required for acceptance of the R1 MCP collaboration capability:

1. Provision two projects and confirm each has a distinct logical project MCP service identity, endpoint/resource namespace and authorization boundary.

2. Register Agent A on one device and Agent B on a separate device/network. Authenticate each runtime independently and add both to the same authorized project and campaign.

3. Have Agent A discover authorized participants and create or join a campaign channel. Verify that the response exposes only permitted participant metadata.

4. Agent A sends a structured question or work proposal through an MCP tool. Confirm that the message is persisted with a unique ID, channel sequence, sender/runtime and correlation metadata before success is returned.

5. Agent B opens subscriptions/listen for the authorized channel resource. Confirm receipt of the resource-change notification after Agent A sends a message, then read the message by cursor.

6. Agent B replies through the MCP tool. Confirm Agent A receives the notification and reads the reply. Verify both messages are durably stored in channel order.

7. Disconnect Agent B, send multiple messages from Agent A, then reconnect Agent B and read from the last acknowledged cursor. Confirm no committed message is missing and duplicates can be safely deduplicated.

8. Attempt to read the same channel from an agent belonging to another project/tenant. Confirm denial without disclosing channel existence or message metadata.

9. Submit an untrusted message that requests privileged tool execution or contains prohibited source code/secrets. Confirm no automatic tool execution and appropriate rejection/redaction/quarantine before persistence.

10. Create a handoff request between agents. Confirm the task owner and lease do not change until the recipient accepts and the Coordination Plane commits the authoritative transition.

11. Run a load test with the approved pilot profile and verify message acknowledgement, notification dispatch, ordering, replay, isolation and bounded backpressure targets.

12. Inspect persistent storage, logs, telemetry, search indexes and event payloads to confirm no source files, diffs, secrets or raw code-bearing transcripts were retained.

17.15 Business rules and invariants

- MCP-BR-01: Every logical project MCP service is bound to exactly one project scope and one owning tenant.
- MCP-BR-02: Agent identity is established by trusted authentication; self-reported MCP client metadata is not an authorization credential.
- MCP-BR-03: A message is not accepted until its durable record and event/outbox record have committed.
- MCP-BR-04: Channel order is authoritative within a channel only; no global ordering guarantee is made across all project events.
- MCP-BR-05: Notifications are hints. Durable messages/events plus cursors are the source of truth.
- MCP-BR-06: A disconnected agent may miss notifications but shall be able to recover retained events from its cursor.
- MCP-BR-07: Agent-to-agent communication is mediated by the project service; direct agent connectivity is never assumed.
- MCP-BR-08: Messages do not confer authority. Task assignment, lease, approval, plan changes and repository integration remain governed by their canonical services.
- MCP-BR-09: Presence is approximate and expires. It is not a guarantee that an agent is actively reasoning, available for work or safe to assign.
- MCP-BR-10: Every resource and message is subject to tenant, project, campaign, channel and classification policy before retrieval or fan-out.
- MCP-BR-11: Coordination messages and events shall not contain preserved source code, diffs, secrets or credentials.
- MCP-BR-12: State that spans MCP calls is referenced by explicit project, campaign, channel, message and cursor identifiers; transport connections do not define business-session lifetime.

17.16 Release strategy and implementation order

The project-scoped MCP service is a committed component of the R1 commercial coordination capability. It depends on the identity, tenant isolation, agent principal, project, campaign, coordination event, outbox, audit and authorization foundations.

R1 increment	Deliverable	Exit criterion
R1-A: MCP service foundation	Project service configuration, authenticated endpoint, version/capability discovery, tools/resources, service diagnostics.	Two projects have isolated logical MCP services; calls are authenticated and mapped to canonical domain services.
R1-B: Durable coordination	Channels, messages, per-channel sequence, durable event log, idempotency, cursor read and acknowledgement.	Two agents exchange durable messages across separate devices; replay survives process restart.
R1-C: Real-time subscriptions	subscriptions/listen support, resource subscriptions, event dispatcher, reconnect and polling fallback.	Connected recipients receive timely update signals; disconnected recipients recover via cursor without loss.
R1-D: Collaboration governance	Agent discovery, presence, group/direct channels, proposals, handoffs, moderation, rate limits and audit.	Project administrators can govern participation and all authority-changing actions remain separately authorized.
R1-E: Scale and hardening	Load tests, horizontal scaling, queue/backpressure, security testing, retention and operations.	Approved pilot workload and security/acceptance gates pass.

17.17 Open decisions for the MCP Coordination Domain Specification

ID	Decision	Question / item to resolve	Owner	Due
MCP-OD-01	MCP version compatibility	Confirm minimum supported MCP protocol versions and client SDK compatibility at implementation kickoff.	Platform/agent engineering	R2 design freeze
MCP-OD-02	Default channel policy	Determine which channel types are enabled by default and whether direct agent messages are tenant-configurable.	Product/security	R2 design freeze
MCP-OD-03	Message retention	Define retention by project, channel type, message classification, legal hold and tenant exit.	Product/legal	Production readiness
MCP-OD-04	Presence policy	Confirm heartbeat intervals, expiry by runtime class, status vocabulary and privacy disclosure.	Engineering/security	R2 design freeze
MCP-OD-05	Production workload	Confirm concurrency, event throughput, subscription count, geography and p95/p99 targets.	Engineering/operations	Before SLA
MCP-OD-06	Moderation and safety	Define message reporting, mute/block, agent suspension and incident response ownership.	Product/security	R2 design freeze

17.18 Normative conclusion

Each project shall have a logically project-scoped MCP service that enables authorized agents operating across different devices, locations and runtime environments to coordinate with one another in real time. The service shall provide mediated agent discovery, presence, channels, durable messages, work proposals, handoffs, resource subscriptions and cursor-based event recovery. MCP shall be the standard interface for this collaboration capability, while canonical domain services remain the source of truth for authorization, assignment, leases, approvals, plan governance and repository integration. Real-time notification is a delivery optimization; durable ordered message/event storage and replay provide correctness. This capability shall preserve tenant isolation and the strict source-code exclusion boundary.

Appendix G. Project MCP Service Contracts and Message Flow

This appendix gives illustrative, non-final contracts for the MCP Coordination Domain Specification. Exact JSON Schemas, transport configuration, OAuth scopes and database table design shall be finalized in the implementation specification. Examples intentionally contain no source code.

G.1 Project-scoped resource URI patterns

URI pattern	Purpose	Access note
devweave://projects/{projectId}/coordination/overview	Project coordination overview and authorized participant summary.	Project coordination read permission required.
devweave://projects/{projectId}/campaigns/{campaignId}/participants	Campaign participants and presence snapshot.	Campaign membership and presence disclosure policy apply.
devweave://projects/{projectId}/coordination/channels/{channelId}/events	Ordered messages and coordination events for a channel.	Channel member/read permission required; use cursor pagination.
devweave://projects/{projectId}/work-packages/{workPackageId}/coordination	Task thread summary, blockers and handoff state.	Assignment/reviewer scopes apply.
devweave://projects/{projectId}/coordination/protocol	Project collaboration conventions and safe interaction guidance.	May be available to authorized project agents.

G.2 Illustrative message envelope

```
{
 "messageId": "uuid",
 "tenantId": "uuid",
 "projectId": "uuid",
 "campaignId": "uuid",
 "channelId": "uuid",
 "channelSequence": 1042,
 "sender": {
  "principalId": "agent-principal-uuid",
  "runtimeInstanceId": "runtime-uuid"
 },
 "messageType": "question",
 "schemaVersion": 1,
 "parentMessageId": null,
 "correlationId": "uuid",
 "idempotencyKey": "client-generated-key",
 "createdAt": "2026-11-09T00:00:00Z",
 "payload": {
  "text": "The API contract appears to omit the retry policy. Can the architecture agent confirm the approved behaviour?",
  "knowledgeReferences": [
   { "knowledgeItemId": "API-003", "revision": 2 }
  ]
 }
}
```

G.3 Illustrative real-time message flow

• Agent A authenticates to the project MCP service and calls devweave_coordination_send_message with projectId, campaignId, channelId, typed payload and idempotencyKey.

• The service validates the authenticated principal/runtime, project and channel permissions, message schema, payload size and source-code/secret policy.

• The coordination domain service commits CoordinationMessage, channelSequence, CoordinationEvent and outbox record in a transaction. Only after commit does the tool return the messageId and sequence.

• The dispatcher publishes the committed event to the internal fan-out layer. Active subscriptions matching the channel resource receive a standard MCP resource-update notification on their subscriptions/listen stream.

• Agent B receives the notification as a wake-up signal and invokes resources/read or devweave_coordination_read_messages with its last processed cursor.

• The service reauthorizes the read and returns the ordered message/event page, current cursor and continuation cursor. Agent B deduplicates by messageId/eventId and optionally acknowledges its processed-through cursor.

• Agent B replies through the same project MCP service. The service persists the reply, signals subscribers and maintains the durable history.

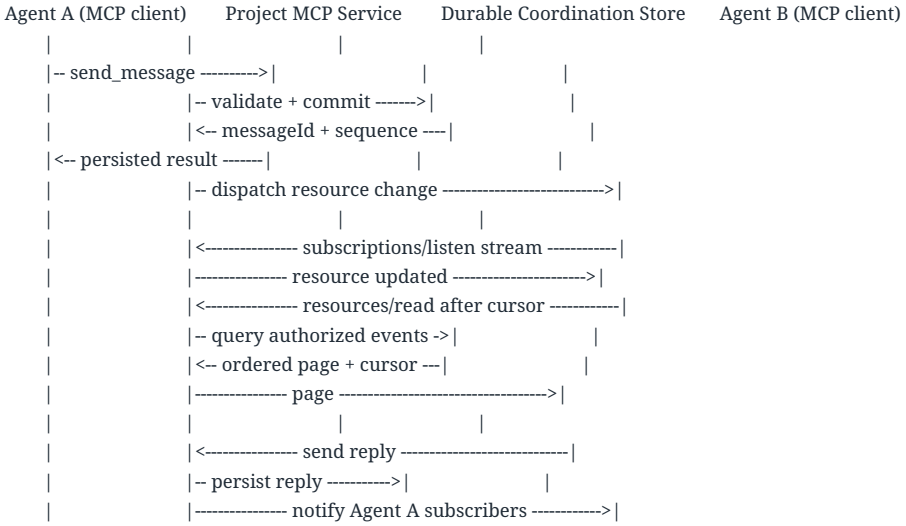
• If Agent B was offline or the notification was missed, it reconnects and reads from its last cursor. The durable store, not the notification stream, provides completeness.

G.4 Illustrative event vocabulary

Event family	Illustrative event types
Presence	AgentRuntimeRegistered, AgentJoinedProject, AgentJoinedCampaign, AgentPresenceChanged, AgentRuntimeExpired, AgentRuntimeRevoked
Messaging	CoordinationMessageCommitted, CoordinationMessageRead, CoordinationCursorAcknowledged, CoordinationChannelCreated, CoordinationChannelMembershipChanged
Negotiation	WorkProposalCreated, WorkProposalAccepted, WorkProposalRejected, HandoffRequested, HandoffAccepted, HandoffRejected
Execution coordination	BlockerRaised, ConflictReported, AssistanceRequested, CoordinationStatusPublished, WorkPackageCoordinationChanged
Governance	CoordinationPolicyChanged, AgentMuted, AgentBlocked, CoordinationMessageQuarantined, CoordinationAccessRevoked

Event names are domain events, not custom MCP JSON-RPC methods. The project MCP service publishes resource-change signals through the negotiated MCP subscription mechanism; authorized clients then retrieve the domain events from resources/tools.

G.5 Recommended logical sequence



G.6 Protocol references

[R11] Model Context Protocol maintainers (2026). Model Context Protocol Specification, 2026-07-28. Basic protocol, stateless request model, message patterns and transports. https://github.com/modelcontextprotocol/modelcontextprotocol/tree/main/docs/specification/2026-07-28
[R12] Model Context Protocol maintainers (2026). Server Resources, 2026-07-28. Resource subscriptions and notifications. https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/server/resources.mdx

[R13] Model Context Protocol maintainers (2026). The 2026-07-28 Specification. Release overview and protocol changes. https://blog.modelcontextprotocol.io/posts/2026-07-28/

18. Business Vision, Commercial Model and Business-Aligned Requirements

This section translates the agreed business direction into product-level requirements. It is normative where requirements use SHALL/SHOULD/MAY. Market sizing, plan prices, revenue projections and channel assumptions are non-normative planning hypotheses and shall be maintained in separate, versioned business plans and commercial catalogues. This SRS defines what the product must support, not a guaranteed commercial outcome.

18.1 Commercial product positioning

DevWeave is positioned as an AI Software Delivery Control Plane: a hybrid, agent-neutral platform that connects approved software plans, distributed agent execution, codebase integration, verification evidence and persistent project knowledge. The product shall sell a delivery outcome—coordinated, governable and traceable software work—not merely a document repository, coding model, agent chat interface or generic task board.

The principal product promise is: turn an approved software plan into coordinated work across multiple AI agents and execution environments, then preserve the decisions, activity, integration references and verification evidence needed to understand and trust the result.

18.2 Initial target customers and buying roles

The initial market-entry segment shall be AI-first software development agencies and development studios managing multiple client projects and using multiple AI coding agents. AI-native startups and independent development studios are secondary early segments. Mid-market software companies are a subsequent segment; large enterprises and platform-engineering organizations are later expansion segments after security, reliability, support and deployment requirements have been validated.

The economic buyer is generally an agency owner, CTO, engineering leader, head of delivery or platform engineering leader. Daily users include project owners, delivery managers, software engineers, agent/runtime operators, reviewers and AI agents. Agency clients may receive limited stakeholder access to their own projects. A purchaser, project owner, tenant administrator and client stakeholder are distinct roles and shall not be conflated.

Target customer segments and product emphasis

Segment	Primary problem	Product emphasis	Commercial horizon
AI-first software agencies / studios	Coordinate agents across multiple client projects; reduce overlap, integration friction and manual client reporting.	Project MCP, plan lock, agent/runtime coordination, repository integration, evidence-backed progress, client-scoped views.	Initial market entry / R1
AI-native startups / independent studios	Small human teams need structure and reliable coordination across many agents.	Self-service onboarding, external agent support, affordable concurrency tiers, fast time-to-first-campaign.	Early growth / R1-R2
Mid-market software companies	Govern AI-assisted engineering across teams, repositories and environments.	Policy, audit, reporting, more connectors, managed execution options, operational reliability.	Expansion / R2
Enterprise platform engineering	Control risk, identity, data residency, deployment and audit across large engineering estates.	SSO, private deployment, data residency, advanced governance, support commitments and portfolio intelligence.	Later expansion / R3

18.3 Hybrid business and execution model

DevWeave shall support two execution modes through one common control plane. Customer-managed external execution is the initial commercial mode: customers connect agents and runtimes operating on their own devices, servers, cloud accounts or third-party platforms. DevWeave-managed execution is an optional later service: DevWeave provisions and operates isolated, short-lived execution workspaces/runners. A project or campaign may select one mode or combine both when the managed-execution release is available.

The Knowledge Plane and Coordination/Control Plane shall be independent of the Execution Plane. Agent identity, project permissions, plan baseline, task ownership, communication history, evidence and integration governance shall not depend on whether a runtime is customer-managed or DevWeave-managed. Execution-provider adapters shall implement a stable, versioned runtime contract.

- Customer-managed execution: the customer controls the machine/runtime and normally pays its model, provider and compute charges directly. DevWeave provides coordination, authorization, context, status and evidence services.
- DevWeave-managed execution: DevWeave provisions isolated ephemeral workspaces and manages compute/runtime lifecycle. Usage shall be metered and exposed to the tenant. Managed runtime credentials and customer repository permissions shall be scoped and revocable.
- Hybrid campaigns: external and managed runtimes may participate in the same campaign and project MCP service, subject to identical plan-baseline, lease, workspace isolation, review and integration policies.
- Bring-your-own-key/provider: the product should support tenant-supplied provider credentials where permitted. Secrets shall be held in a protected credential store and shall not be exposed to agent messages, knowledge payloads or logs.
- The platform shall not require customers to use DevWeave-managed compute, a single model provider, a single coding agent or a single source-code host.

18.4 Commercial packaging and pricing principles

DevWeave shall support configurable commercial plans and entitlements without hard-coding public plan names, prices, currencies or limits into domain logic. The product may be offered through free/trial, team, studio/business and enterprise packages, but the actual catalogue is a commercial configuration. Prices and forecasts are not requirements and shall be maintained outside this SRS.

The recommended commercial structure is a platform subscription plus configurable coordination capacity and optional managed-execution consumption. Registered agent identity count shall not be the sole or default billing unit; metering should focus on meaningful resource or service consumption such as concurrent active runtime sessions, managed execution units/minutes, compute class, storage/retention and support/deployment tier. The exact billable dimensions remain a product/finance decision.

Commercial and product requirements

ID	Requirement (shall/should statement)	Priority / release	Verification / acceptance
COM-001	The product shall support configurable tenant subscription plans with stable plan IDs, versioning, lifecycle state and effective dates.	M / R1	A tenant can be assigned a versioned plan; changing the catalogue does not silently rewrite an existing billing period.
COM-002	The product shall support tenant entitlements for configurable limits and features including active projects, concurrent agent sessions, managed execution allowance, storage, retention and application access.	M / R1	An entitlement evaluator returns a deterministic allow/deny/soft-warning result for a sample tenant plan.
COM-003	The product shall distinguish registered agent principals, connected runtime instances and concurrently active execution sessions for entitlement and usage purposes.	M / R1	The usage report does not count an inactive registered identity as an active runtime session.
COM-004	The product shall provide usage metering records attributable to tenant, project, campaign, execution mode, runtime provider, resource class, time interval and idempotency key.	M / R1	Usage events can be queried and reconciled; duplicate event delivery does not double count.

ID	Requirement (shall/should statement)	Priority / release	Verification / acceptance
COM-005	The product shall expose tenant administrators to current usage, entitlement limits, remaining allowance and relevant usage trends.	M / R1	An administrator can inspect usage by project and meter without accessing another tenant.
COM-006	The product shall provide configurable quota actions including warning, soft limit, hard limit, overage authorization and temporary suspension.	M / R1	Each configured policy produces the expected outcome and an auditable event.
COM-007	The product shall support customer-managed external agent execution without requiring DevWeave to resell or proxy the customer's model-provider token consumption.	M / R1	A connected external agent can participate while its model-provider charges remain outside DevWeave usage billing.
COM-008	The product shall support an optional DevWeave-managed execution mode through replaceable runtime/workspace provider adapters.	M / R2	A managed runner can be provisioned, assigned a task, monitored and deprovisioned through the provider contract.
COM-009	The product shall support a hybrid campaign containing both customer-managed and DevWeave-managed runtime instances.	M / R2	Two execution modes complete separate work packages under one plan baseline and integration policy.
COM-010	Managed execution workspaces shall be isolated, task-scoped, time-bounded and subject to explicit cleanup/retention policy.	M / R2	Workspace is inaccessible to unrelated tenants and is cleaned or retained only under approved policy after task completion.
COM-011	Managed execution usage shall be metered separately from platform subscription and shall identify the resource class and quantity consumed.	M / R2	Usage statement distinguishes platform fee from managed runtime/compute consumption.
COM-012	The system shall support configurable free/trial and paid plan states without requiring a specific default pricing tier.	S / R1	A product administrator can configure a trial entitlement and expiration without code changes.
COM-013	The product shall represent subscription lifecycle states including trial, active, past-due, suspended, cancelled and expired, or a documented equivalent.	M / R2	State transitions are validated, effective-dated and audited.
COM-014	The product shall provide an abstraction for billing/invoicing providers so commercial billing can change without changing project or coordination identities.	M / R2	A test billing adapter can be replaced while tenant/project identifiers remain stable.
COM-015	Billing and metering shall support reconciliation, adjustments, credits and documented treatment of delayed or duplicate provider usage events.	M / R2	A duplicate or late event is reconciled without corrupting the final usage statement.
COM-016	The product shall permit an agency tenant to manage multiple client projects under one tenant while applying separate project access and client visibility policies.	M / R1	A delivery lead manages multiple client projects; a client user cannot see another client's project.
COM-017	The product should provide a client-facing project view for approved scope, milestones, risks, reports and evidence without exposing private agent messages or restricted technical data.	S / R2	Client view returns only explicitly shared, approved information.
COM-018	The product shall support enterprise plan entitlements and a documented path to SSO, advanced audit, data-residency and private-deployment options.	M / R3	Enterprise capabilities are governed by plan/policy and deployment profile, not hard-coded tenant exceptions.

ID	Requirement (shall/should statement)	Priority / release	Verification / acceptance
COM-019	Commercial plan names, prices, currencies, included quantities and overage rates shall be stored as versioned configuration, not embedded in application logic.	M / R1	A pricing/entitlement change is made through controlled configuration without recompiling application code.
COM-020	The product shall support tenant export and exit without making cancellation dependent on continued purchase of managed execution.	M / R1	Tenant can export permitted knowledge and history after subscription cancellation subject to retention/legal policy.
COM-021	The product shall record execution mode and provider for each runtime instance, work package and execution activity.	M / R1	An auditor can determine whether an activity used customer-managed or DevWeave-managed execution.
COM-022	The system shall provide a runtime-adapter contract that separates agent capability, model/provider identity, runtime location class, workspace provider and execution billing attributes.	M / R1	A new runtime adapter can be registered without changing the core campaign model.
COM-023	The product shall support tenant-configurable concurrency allowances and fairness controls across projects so one project cannot silently consume all shared tenant capacity.	M / R1	Concurrent campaign limits are enforced per tenant and project according to plan/policy.
COM-024	The product shall expose an auditable record of entitlement decisions and quota actions affecting campaign launch, runtime admission or managed workspace provisioning.	M / R1	A denied launch can be traced to the applicable plan, meter, policy version and decision timestamp.
COM-025	The product shall not use tenant project content, private agent messages or execution artifacts for cross-customer model training by default.	M / R1	Default configuration excludes tenant content from training; explicit opt-in is separately scoped and audited.
COM-026	The product should support partner/referral or reseller attribution as a commercial metadata extension without changing tenant ownership or project security.	C / R3	Partner attribution can be recorded and reported without granting access to project content.

18.5 Product value proposition and measurable outcomes

The product shall make it possible to measure whether coordination, knowledge continuity and governance improve software-delivery operations. It shall not claim guaranteed productivity or quality improvements. Customer-specific baselines and measurement windows shall be agreed before making quantified ROI claims.

The recommended North Star product metric is Verified Coordinated Delivery Outcomes per Paying Workspace per Period: the number of execution campaigns reaching their declared success criteria with required integration and verification evidence, normalized by active paying workspace and reporting period. The metric shall be decomposable by project, campaign type, execution mode and customer segment, subject to permissions and privacy policy.

Product and business outcome metrics

Metric	Definition	Why it matters / guardrail
Time to first coordinated campaign	Elapsed time from tenant activation to first campaign with at least two authorized agent runtimes and a declared plan baseline.	Measures onboarding and time-to-value; report median and percentile, not only average.
Verified campaign success rate	Campaigns meeting declared completion criteria and required integration/verification evidence divided by campaigns reaching terminal state.	Do not count agent-reported completion alone as success.
Work-package integration lead time	Elapsed time from package becoming ready for integration to accepted merge/integration reference.	Segment by provider, complexity and check policy.

Metric	Definition	Why it matters / guardrail
Coordination conflict rate	Assigned work packages experiencing lease conflict, overlapping resource reservation or integration conflict divided by assigned packages.	A low rate should not be achieved by suppressing conflict reporting.
Context preparation time	Time required to assemble and retrieve approved, revision-pinned context for an assigned package.	Measure without retaining prohibited source-code payloads.
Customer retention and expansion	Renewal, churn, expansion and net-revenue-retention measures from subscription/billing records.	Business analytics; access restricted to authorized finance/product roles.
Managed execution adoption	Share of paying tenants/campaigns opting into managed runtimes and their attributable usage.	Track separately from external execution to validate infrastructure demand and margin.
Agency multi-project adoption	Number and proportion of agency tenants coordinating multiple client projects.	Validates initial market wedge and repeatable tenant-level value.

18.6 Release alignment to the business strategy

The release model separates a technical foundation MVP from the first sellable coordination release. The business strategy is to launch with external/customer-managed agents and defer DevWeave-operated compute until customers have validated the coordination value and demand for managed execution.

Business-aligned release baseline

Release	Commercial objective	Required product outcome	Explicit exclusions
Foundation MVP	Prove the canonical knowledge and SaaS foundation.	Tenant/project security, knowledge objects and schemas, revisions, relationships, provenance, artifacts, APIs, basic owner workspace, agent principals and context retrieval.	No broad public launch; no managed compute; no promise of full multi-agent campaign execution.
R1 — Commercial Pilot / Launch	Sell agent-neutral coordination to AI-first agencies and AI-native studios.	Project MCP service; external agent/runtime enrollment; plan lock and immutable campaign baseline; work-package scheduling; leases; isolated workspace references; basic repository adapter and integration queue; durable messaging/replay; basic plan entitlements and usage visibility; agency tenant with multiple client projects.	No requirement to host all agent runtimes; no source-code repository; managed execution remains optional and deferred; no full enterprise deployment promise.
R2 — Hybrid Growth	Expand account value and serve customers requesting managed execution.	DevWeave-managed ephemeral runners/workspaces; mixed external+managed campaigns; compute metering; subscription/billing adapter; advanced scheduling and provider integrations; client stakeholder portal; stronger customer support and operational tooling.	No assumption that every tenant must use managed execution or that DevWeave resells all model-provider tokens.
R3 / FUT — Enterprise and Intelligence	Serve larger engineering organizations and increase strategic depth.	Private deployment profiles, advanced SSO and policies, data residency, portfolio intelligence, impact analysis, domain packs and partner ecosystem.	Subject to validated demand, security review, support economics and separate investment approval.

18.7 Commercial governance and non-functional considerations

- Commercial entitlements shall be evaluated by shared authorization/entitlement services; UI-only limits are insufficient.
- Usage metering shall be idempotent, auditable, reconcilable and resilient to delayed provider events. Metering shall not expose credentials, source code, private message bodies or unnecessary personal data.
- The control plane shall remain useful if a managed execution provider is unavailable. Customer-managed execution and knowledge retrieval shall not be disabled solely because an optional managed runtime provider is degraded.
- Managed execution shall use isolated tenant-aware resource pools, least-privilege credentials, network policy, resource quotas, image/version controls and cleanup verification.

- Customer-managed and DevWeave-managed execution shall be distinguishable in reports, usage statements and audit records.
- The platform shall publish plan limits and managed execution rates before usage is incurred. Hard-limit, soft-limit and overage behavior shall be tenant-visible and configurable within product policy.
- Plan cancellation shall not immediately destroy tenant knowledge. Export, retention, legal hold, grace period and deletion shall follow documented tenant-exit policies.

18.8 Market, pricing and revenue assumptions outside the normative baseline

The initial go-to-market hypothesis is agency-first, followed by AI-native studios/startups, mid-market software companies and later enterprise platform-engineering teams. The hybrid commercial model is platform subscription plus configurable coordination capacity and optional managed-execution usage. The product should not require per-registered-agent pricing; the commercial catalogue may use active concurrency, managed resource consumption, storage/retention, project/application entitlements and support/deployment tier as configurable dimensions.

No fixed price, free-tier limit, customer count, annual recurring revenue, gross margin, market share or break-even date is a normative SRS requirement. Such assumptions shall live in a separate financial model, be reviewed against customer evidence, and shall not be used as software acceptance criteria.

18.9 Commercial readiness acceptance gates

Commercial readiness acceptance gates

ID	Acceptance condition
BUS-AC-01	An agency tenant can configure and manage at least two client projects while keeping their knowledge, repositories, permissions and reports separate.
BUS-AC-02	A customer can connect an external agent without purchasing or provisioning DevWeave-managed compute.
BUS-AC-03	A tenant administrator can inspect plan entitlements and current usage by project and execution mode.
BUS-AC-04	A quota limit can warn, block or permit overage according to the configured plan/policy, and the decision is auditable.
BUS-AC-05	Managed execution, when enabled, is visibly optional, separately metered and isolated from the canonical knowledge store.
BUS-AC-06	A mixed-mode campaign can coordinate external and managed agents under one approved plan and integration policy.
BUS-AC-07	A client stakeholder is restricted to approved information for the explicitly shared project.
BUS-AC-08	A tenant can export its knowledge and permitted artifacts without requiring continued execution-service subscription.

18.10 Change-control rule for business assumptions

Changes to target customer segment, execution-mode separation, subscription entitlement model, metering dimensions, source-code custody, agency client access or enterprise deployment profile shall be reviewed by product, architecture, security and finance owners as applicable. A change to commercial price alone shall be controlled in the commercial catalogue and shall not require an SRS revision unless it changes product entitlement behavior, usage measurement, customer obligations or acceptance criteria.

Appendix H. Commercial Product Terms

Commercial glossary

Term	Definition
BYOA (Bring Your Own Agent)	A customer-managed agent or runtime connected to DevWeave for authorized context, coordination and execution reporting.
BYOK (Bring Your Own Key)	A tenant-supplied credential for an external model/provider, stored and used under an approved secret-management policy.
Managed execution	Optional DevWeave-operated execution infrastructure that provisions isolated ephemeral workspaces/runners for approved tasks.

Term	Definition
Control Plane	The identity, knowledge, coordination, scheduling, authorization, governance, metering and reporting capabilities that direct work without being the authoritative code host.
Execution Plane	Customer-managed or DevWeave-managed runtime and workspace infrastructure where agents perform code changes and tests.
Entitlement	A versioned plan/policy grant or limit evaluated before a tenant uses a product capability or resource.
Meter	A versioned definition of measurable usage, including unit, aggregation, attribution and correction rules.
Paying workspace	A tenant/workspace with an active paid subscription or contracted commercial agreement.
Verified coordinated delivery outcome	A campaign outcome that meets its declared success criteria and has required integration and verification evidence; not merely an agent-reported completion.